

Key Management Interoperability Protocol Usage Guide Version 3.0

Committee Note Draft 01

18 June 2026

This stage:

<https://docs.oasis-open.org/kmip/kmip-ug/v3.0/cnd01/kmip-ug-v3.0-cnd01.docx> (Authoritative)
<https://docs.oasis-open.org/kmip/kmip-ug/v3.0/cnd01/kmip-ug-v3.0-cnd01.html>
<https://docs.oasis-open.org/kmip/kmip-ug/v3.0/cnd01/kmip-ug-v3.0-cnd01.pdf>

Previous stage:

<https://docs.oasis-open.org/kmip/kmip-ug/v2.1/cn01/kmip-ug-v2.1-cn01.docx> (Authoritative)
<https://docs.oasis-open.org/kmip/kmip-ug/v2.1/cn01/kmip-ug-v2.1-cn01.html>
<https://docs.oasis-open.org/kmip/kmip-ug/v2.1/cn01/kmip-ug-v2.1-cn01.pdf>

Latest stage:

<https://docs.oasis-open.org/kmip/kmip-ug/v3.0/kmip-ug-v3.0.docx> (Authoritative)
<https://docs.oasis-open.org/kmip/kmip-ug/v3.0/kmip-ug-v3.0.html>
<https://docs.oasis-open.org/kmip/kmip-ug/v3.0/kmip-ug-v3.0.pdf>

Technical Committee:

OASIS Key Management Interoperability Protocol (KMIP) TC

Chairs:

Judith Furlong (Judith.Furlong@dell.com), Dell Technologies
Tim Hudson (tjh@cryptsoft.com), Cryptsoft Pty Ltd

Editors:

Judith Furlong (Judith.Furlong@dell.com), Dell Technologies
Jeff Bartell (jeff@bartellemail.com), Individual Member

Related work:

This document replaces or supersedes:

- *Key Management Interoperability Protocol Usage Guide Version 2.1*. Edited by Judith Furlong. Latest stage: <https://docs.oasis-open.org/kmip/kmip-ug/v2.1/kmip-ug-v2.1.html>.

This document is related to:

- *Key Management Interoperability Protocol Specification Version 3.0*. Edited by Greg Scott and Charles White. Latest stage: <https://docs.oasis-open.org/kmip/kmip-spec/v3.0/kmip-spec-v3.0.html>.
- *Key Management Interoperability Protocol Profiles Version 3.0*. Edited by Tim Chevalier and Tim Hudson. Latest stage: <https://docs.oasis-open.org/kmip/kmip-profiles/v3.0/kmip-profiles-v3.0.html>.

Abstract:

This document is intended for developers and architects who wish to design systems and applications that interoperate using the Key Management Interoperability Protocol specification.

Status:

This is a Non-Standards Track Work Product. The patent provisions of the OASIS IPR Policy do not apply.

This document was last revised or approved by the OASIS Key Management Interoperability Protocol (KMIP) TC on the above date. The level of approval is also listed above. Check the “Latest stage” location noted above for possible later revisions of this document. Any other numbered Versions and other technical work produced by the Technical Committee (TC) are listed at https://www.oasis-open.org/committees/tc_home.php?wg_abbrev=kmip#technical.

TC members should send comments on this document to the TC's email list. Others should send comments to the TC's public comment list, after subscribing to it by TC members should send comments on this specification to the TC's email list. Others should send comments to the TC's public comment list, after subscribing to it by following the instructions on the TC's web page at <https://groups.oasis-open.org/communities/community-home?CommunityKey=2b5e5c66-cc41-4aa5-92ee-018f5aa7dfc4>.

Notices:

Copyright © OASIS Open 2026. All Rights Reserved.

Distributed under the terms of the OASIS IPR Policy, [<https://www.oasis-open.org/policies-guidelines/ipr/>]. For complete copyright information please see the Notices section in the Appendix.

Table of Contents

1	Introduction	6
1.1	References (non-normative)	6
2	Assumptions	9
2.1	Island of Trust	9
2.2	Message Security	9
2.3	Extensible Protocol	9
2.4	Server Policy	9
2.5	Synchronous and Asynchronous Operations	9
2.6	Support for “Intelligent Clients” and “Key Using Devices”	9
2.7	Batched Requests and Responses	9
2.8	Reliable Message Delivery	10
2.9	Large Responses	10
2.10	Key Life-cycle and Key State	10
3	Using KMIP Functionality	11
3.1	Authentication	11
3.1.1	Credential	11
3.1.2	Device Credential Type	12
3.2	Additional Authorization for Certain Operations	13
3.3	Returning Related Objects	14
3.4	Compromised Objects	14
3.5	ID Placeholder	14
3.6	Use of Meta-Data Only (MDO) Keys	15
3.7	Using the Same Asymmetric Key Pair in Multiple Algorithms	16
3.8	Non-Cryptographic Objects	16
3.9	Split Key	16
3.10	Asymmetric Concepts with Symmetric Keys	17
3.11	Key Rotation	18
3.12	Key Block	18
3.13	Key Encoding	19
3.13.1	AES Key Encoding	19
3.13.2	Triple-DES Key Encoding	19
3.13.3	AES-XTS Key Encoding	19
3.14	Attributes	20
3.15	Key State and Times	20
3.16	Application Specific Information	21
3.17	Description and Comment Attributes	22
3.18	Cryptographic Length of Asymmetric Keys	22
3.19	Constraints	23
3.20	Multi-instance Hash	23
3.21	Extractable and Sensitive Attributes	23
3.22	Key Format Type	23

3.23 Using the “Raw” Key Format Type	23
3.24 PKCS#12 Key Format	23
3.25 Protection Storage Mask	24
3.26 Unique Identifiers	24
3.27 Usage Allocation	25
3.28 Canceling Asynchronous Operations	25
3.29 Full Async	25
3.30 Querying Outstanding Asynchronous Requests	25
3.31 Process	25
3.32 Cryptographic Services	26
3.32.1 Streaming Cryptographic Services	26
3.32.2 Security Considerations for Server State Handling.	26
3.33 Archive Operations	27
3.34 Certify and Re-certify	27
3.35 Using Offset in Re-key and Re-certify Operations	27
3.36 Certificate Renewal, Update, and Re-key	27
3.37 Certificate Revocation Lists	28
3.38 Delegated Login	28
3.39 Discover Versions	28
3.40 Interop	28
3.41 Login and Logout	28
3.42 Query	29
3.42.1 Query Function	29
3.43 Registering a Key Pair	29
3.44 Using Notify and Put Operations	29
3.45 Flow Control	30
3.46 Client and Server Correlation Values	30
3.47 Reducing Multiple Requests Through the Use of Batch	30
3.48 Maximum Message Size	30
3.49 Message Extensions	31
3.50 Result Message Text	31
3.51 Result Reasons	31
3.52 Passing Attestation Data	31
3.53 Vendor Extensions	32
3.54 Counters	32
3.55 PKCS#11	32
3.56 Post Quantum Cryptography	33
4 Applying KMIP Functionality	34
4.1 Locating Keys in Specific States	34
4.2 Using Wrapped Keys with KMIP	35
4.2.1 Encrypt-only Example with a Symmetric Key as an Encryption Key for a Get Request and Response	36

4.2.2 Encrypt-only Example with a Symmetric Key as an Encryption Key for a Register Request and Response	36
4.2.3 Encrypt-only Example with an Asymmetric Key as an Encryption Key for a Get Request and Response	37
4.2.4 MAC-only Example with an HMAC Key as an Authentication Key for a Get Request and Response	37
4.2.5 Registering a Wrapped Key as an Opaque Cryptographic Object	38
4.2.6 Encoding Option for Wrapped Keys	38
4.3 Interoperable Key Naming for Tape	39
4.3.1 Native Tape Encryption by a KMIP Client	39
4.4 Registering Extension Information	42
4.5 Using KMIP for PGP Keys	43
4.6 KMIP Client Registration Models	43
4.6.1 Manual Client Registration	44
4.6.2 Automated Client Registration	44
4.6.3 Registering Sub-Clients Based on a Trusted Primary Client	45
4.7 Using One Time Pad Algorithms	45
4.8 Cryptographic Erasure and Key Shredding	45
4.9 ReEncrypt	46
4.10 Key Replication Between Servers	46
4.10.1 Architecture Considerations	46
5 Deprecated KMIP Functionality	47
5.1 KMIP Deprecation Rule	47
5.2 Deprecated Functions	47
Appendix A. Acknowledgments	48
Appendix B. Table of Figures and Tables	50
Appendix C. Revision History	51

1 Introduction

This Key Management Interoperability Protocol Usage Guide Version 3.0 is intended to complement the Key Management Interoperability Protocol Specification [KMIP-Spec] by providing guidance on how to implement the Key Management Interoperability Protocol (KMIP) most effectively to ensure interoperability and to address key management usage scenarios. It includes the following guidance:

- Clarification of assumptions and requirements that drive or influence the design of KMIP and the implementation of KMIP-compliant key management.
- Specific recommendations for implementation of KMIP functionality.
- Clarification of mandatory and optional capabilities for conformant implementations.
- Descriptions of how to use KMIP functionality to address specific key management usage scenarios or to solve key management related issues.

A selected set of conformance profiles and authentication suites are defined in the KMIP Profiles specification [KMIP-PROF].

Further assistance for implementing KMIP is provided by the KMIP Test Cases document [KMIP-TC] that describes a set of recommended test cases and provides the TTLV (Tag/Type/Length/Value) format for the message exchanges defined by those test cases.

1.1 References (non-normative)

NOTE (remove this note and following examples before publication):

Recommended approach: Set up [Reference] label elements as "Bookmarks", then use hyperlinks to them within the document. (**Here's how:** Insert hyperlink ->Place in this document -> scroll down to Bookmarks, select appropriate one.)

Use the "Ref" paragraph style to format references.

The proper format for citation of technical work produced by an OASIS TC (whether Standards Track or Non-Standards Track) is:

[Citation Label] Work Product [title](#) (italicized). Edited by Albert Alston, Bob Ballston, and Calvin Carlson. Approval date (DD Month YYYY). OASIS [Stage](#) Identifier and [Revision](#) Number (e.g., OASIS Committee Specification Draft 01). Principal URI ([version-specific URI](#), e.g., with stage component: somespec-v1.0-csd01.html). Latest version: ([latest version URI](#), without stage identifiers).

For example:

[OpenDoc-1.2] *Open Document Format for Office Applications (OpenDocument) Version 1.2*. Edited by Patrick Durusau and Michael Brauer. 19 January 2011. OASIS Committee Specification Draft 07. <http://docs.oasis-open.org/office/v1.2/csd07/OpenDocument-v1.2-csd07.html>. Latest version: <http://docs.oasis-open.org/office/v1.2/OpenDocument-v1.2.html>.

Reference sources:

For references to IETF RFCs, use the approved citation formats at: <http://docs.oasis-open.org/templates/ietf-rfc-list/ietf-rfc-list.html>.

For references to W3C Recommendations, use the approved citation formats at: <http://docs.oasis-open.org/templates/w3c-recommendations-list/w3c-recommendations-list.html>.

[ECC-Brainpool] *ECC Brainpool Standard Curves and Curve Generation v. 1.0.19.10.2005*, <https://std.neuromancer.sk/brainpool>

- [IEEE 2883-2022] *IEEE Standard for Sanitizing Storage*, IEEE 2883-2022, August 17, 2022, <https://standards.ieee.org/ieee/2883/10277/>
- [KMIP-SPEC] *Key Management Interoperability Protocol Specification Version 3.0*. Edited by Greg Scott and Charles White. Latest version: <https://docs.oasis-open.org/kmip/kmip-spec/v3.0/kmip-spec-v3.0.html>.
- [KMIP-PROF] *Key Management Interoperability Protocol Profiles Version 3.0*. Edited by Tim Hudson and Tim Chevalier. Latest version: <https://docs.oasis-open.org/kmip/kmip-profiles/v3.0/kmip-profiles-v3.0.html>.
- [KMIP-TC] *Key Management Interoperability Protocol Test Cases Version 3.0*. Edited by Tim Chevalier and Tim Hudson. Latest version: <https://docs.oasis-open.org/kmip/kmip-testcases/v3.0/kmip-testcases-v3.0.html>.
- [FIPS 180-4] *Secure Hash Standard (SHS)*, FIPS PUB 180-4, August 2015, <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>
- [FIPS 186-5] *Digital Signature Standard (DSS)*. FIPS PUB 186-4. February 2023. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-5.pdf>
- [FIPS 197-upd1] *Advanced Encryption Standard (AES)*. FIPS PUB 197-upd1. May 9, 2023. <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf>
- [FIPS 198-1] *The Keyed-Hash Message Authentication Code (HMAC)*. FIPS PUB 198-1. July 2008. http://csrc.nist.gov/publications/fips/fips198-1/FIPS-198-1_final.pdf
- [PKCS#11] OASIS PKCS#11 Cryptographic Token Interface Base Specification Version 3.1 <https://docs.oasis-open.org/pkcs11/pkcs11-spec/v3.1/os/pkcs11-spec-v3.1-os.pdf>
- [RFC1321] R. Rivest, *The MD5 Message-Digest Algorithm*, IETF RFC 1321, Apr 1992, <https://www.ietf.org/rfc/rfc1321.txt>
- [RFC1421] J. Linn, *Privacy Enhancement for Internet Electronic Mail: Part I: Message Encryption and Authentication Procedures*, IETF RFC 1421, Feb 1993, <https://www.ietf.org/rfc/rfc1421.txt>
- [RFC2986] M. Nystrom and B. Kaliski, *PKCS#10: Certification Request Syntax Specification Version 1.7*, RFC2986, November 2000, <https://www.ietf.org/rfc/rfc2986>
- [RFC3647] S. Chokhani, W. Ford, R. Sabett, C. Merrill, and S. Wu. *RFC3647: Internet X.509 Public Key Infrastructure Certificate Policy and Certification Practices Framework*. November 2003. <https://www.ietf.org/rfc/rfc3647.txt>
- [RFC4210] C. Adams, S. Farrell, T. Kause and T. Mononen, *Internet X.509 Public Key Infrastructure Certificate Management Protocol (CMP)*, IETF RFC 2510, Sep 2005, <https://www.ietf.org/rfc/rfc4210.txt>
- [RFC4211] J. Schaad, *Internet X.509 Public Key Infrastructure Certificate Request Message Format (CRMF)*, IETF RFC 4211, Sep 2005, <https://www.ietf.org/rfc/rfc4211.txt>
- [RFC4949] R. Shirey. *RFC4949: Internet Security Glossary, Version 2*. August 2007. <https://www.ietf.org/rfc/rfc4949.txt>
- [RFC4880] J. Callas, L. Donnerhacke, H. Finney, D. Shaw and R. Thayer. *RFC4880: OpenPGP Message Format. November 2007*. <https://www.ietf.org/rfc/rfc4880.txt>
- [RFC5272] J. Schaad and M. Meyers, *Certificate Management over CMS (CMC)*, IETF RFC 5272, Jun 2008, <https://www.ietf.org/rfc/rfc5272.txt>
- [RFC5280] D. Cooper, S. Santesson, S. Farrell, S. Boeyen, R. Housley, and W. Polk. *RFC5280: Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*. May 2008. <https://www.ietf.org/rfc/rfc5280.txt>
- [RFC5639] M. Lochter, J. Merkle, *Elliptic Curve Cryptography (ECC) Brainpool Standard Curves and Curve Generation*, IETF RFC 5639, March 2010, <https://www.ietf.org/rfc/rfc5639.txt>
- [RFC7292] K. Moriarty, M. Nystrom, S. Parkinson, A. Rusch, M. Scott, PKCS#12: Personal Information Exchange Syntax v1.1, IETF RFC 7292, July 2014, <https://www.ietf.org/rfc/rfc7292>
- [RFC7748] Langley, A., Hamburg, M., and S. Turner, "Elliptic Curves for Security", RFC 7748, DOI 10.17487/RFC7748, January 2016, <https://www.ietf.org/rfc/rfc7748>

- [RFC8017] K. Moriarty, B. Kaliski, J. Jonsson and A. Rusch, *PKCS#1: RSA Cryptography Specifications* Version 2.2, RFC 8017, November 2016, <https://www.ietf.org/rfc/rfc8017>
- [RFC9562] K. Davis, B. Peabody, P. Leach, *Universally Unique IDentifiers (UUIDs)*, IETF RFC 9662, May 2024, <https://www.ietf.org/rfc/rfc9562>
- [SAM-TSS] *SAM Threshold Sharing Schemes Version 1.0*, Edited by Tim Chevalier and Tim Hudson. Latest versions: <http://docs.oasis-open.org/sam/sam-tss/v1.0/sam-tss-v1.0.html>.
- [SEC2] SEC 2: Recommended Elliptic Curve Domain Parameters, <http://www.secg.org/SEC2-Ver-1.0.pdf>.
- [SP800-38A] M. Dworkin. *Recommendation for Block Cipher Modes of Operation – Methods and Techniques*. NIST Special Publication 800-38A, Dec 2001. <http://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38a.pdf>
- [SP800-38D] M. Dworkin. *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. NIST Special Publication 800-38D. Nov 2007. <http://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38d.pdf>.
- [SP800-38E] M. Dworkin, *Recommendation for Block Cipher Modes of Operation: The XTS-AES Mode for Confidentiality on Block-Oriented Storage Devices*, NIST Special Publication 800-38E, January 2010, <http://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-38e.pdf>.
- [SP800-56A] E. Barker, L. Chen, A. Roginsky, Apostol Vassilev and Richard Davis, *Recommendations for Pair-Wise Key Establishment Schemes Using Discrete Logarithm Cryptography*, NIST Special Publication 800-56A Revision 3, April 2018, <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-56Ar3.pdf>
- [SP800-57-1] E. Barker, *Recommendations for Key Management – Part 1: General (Revision 5)*, NIST Special Publication 800-57 Part 1 Revision 5, May 2020, <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r5.pdf>
- [SP800-88r2] R. Chandramouli and E. A. Hibbard, , *Guidelines for Media Sanitization*, NIST Special Publication 800-88 Revision 2, September 2025, <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-88r2.pdf>
- [SP800-224] Meltem Sonmez Turan and Luis T.A.N. Brandao, *Keyed-Hash Message Authentication Code (HMAC)*, NIST Special Publication 800-224 ipd, June 2024, <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-224.ipd.pdf>
- [X.509] International Telecommunications Union (ITU)-T, *X.509: Information technology – Open systems interconnection – The Directory: Public-key and attribute certificate frameworks*, October 2019, <https://handle.itu.int/11.1002/1000/14033>
- [X9.42] ANSI, *X9.42: Public Key Cryptography for the Financial Services Industry: Agreement of Symmetric Keys Using Discrete Logarithm Cryptography*. 2003. <https://webstore.ansi.org/standards/ascx9/ansix9422003r2013>
- [X9.142] ANSI, *X9.142-2020: Public Key Cryptography for the Financial Services Industry, The Elliptic Curve Digital Signature Algorithm (ECDSA)*, 2020. <https://webstore.ansi.org/standards/ascx9/ANSIX91422020>

2 Assumptions

The section describes assumptions that underlie the KMIP protocol and the implementation of clients and servers that utilize the protocol.

2.1 Island of Trust

Clients may be provided key material by the server, but they only use that key material for the purposes explicitly listed in the Cryptographic Usage Mask. Clients that ignore these instructions and use the keys in ways not explicitly allowed by the server are non-compliant. There is no requirement for the key management system, however, to enforce this behavior.

2.2 Message Security

KMIP relies on the chosen authentication suite as specified in [\[KMIP-PROF\]](#) to authenticate the client and on the underlying transport protocol to provide confidentiality, integrity, message authentication and protection against replay attack..

2.3 Extensible Protocol

The protocol provides for “private” or vendor-specific extensions, which allow for differentiation among vendor implementations. However, any objects, attributes and operations included in an implementation are always implemented as specified in [\[KMIP-SPEC\]](#), regardless of whether they are optional or mandatory.

2.4 Server Policy

A server is expected to be conformant to KMIP and supports the conformance clauses as specified in [\[KMIP-PROF\]](#). However, a server may refuse a server-supported operation or client-settable attribute if disallowed by the server policy (whether expressed within or outside KMIP). Such a decision by the server may reflect the trust relationship with a particular client, performance impact of the requested operation, or any of a number of other considerations.

2.5 Synchronous and Asynchronous Operations

The protocol allows two modes of operation: synchronous and asynchronous. Synchronous operations are those in which a client sends a request and waits for a response from the server. Asynchronous operations (are those in which the client sends a request, the server responds with a “pending” status, and the client polls the server for the completed response and completion status. Server implementations must support synchronous operations but may choose not to support asynchronous operations. Clients may explicitly request operations to be performed asynchronously or may specify that the server not process a request in an asynchronous manner. Servers are required to return whether they support asynchronous processing in the Capability Information from a Query response. Operations are provided to return the list of currently pending requests, and to process immediately (synchronously) a pending request.

2.6 Support for “Intelligent Clients” and “Key Using Devices”

The protocol supports intelligent clients, such as end-user workstations, which are capable of requesting all of the functions of KMIP. It also allows subsets of the protocol in order to support less-capable devices, which only need a subset of the features of KMIP.

2.7 Batched Requests and Responses

The protocol contains a mechanism for sending batched requests and receiving the corresponding batched responses, to allow for higher throughput on operations that deal with a large number of entities,

e. g., requesting dozens or hundreds of keys from a server at one time, and performing operations in a group. A Batch Error Continuation option is provided to indicate whether to undo all previous successful operations once a request in the batch fails; to continue processing requests after an earlier request in the batch fails; or to stop processing the remaining requests in the batch (but not undo previously successful operations). Information may be pipelined between one or more batch items using either the ID Placeholder mechanism or via an indirect reference using an enumerated type or an integer to indicate explicitly the origin of the value.

2.8 Reliable Message Delivery

The reliable message delivery function is relegated to the transport protocol and is not part of the key management protocol itself.

2.9 Large Responses

For requests that could result in large responses, a mechanism in the protocol allows a client to specify in a request the maximum allowed size of a response, or in the case of the Locate operation, the maximum number of items (Maximum Items) which should be returned. If specified limits in a request are exceeded, the server responds indicating that the response would have been too large and does not return any response content.

There is also an offset (Offset Items) parameter that can be set in the Locate operation which allows for “paging” through a large set of results and a Located Items result field that provides how many items could have been returned if there was not limit on the number of items returned within the request.

The Maximum Items and Offset Items operate together to provide the typical SQL offset-and-limit mechanism to KMIP. With the knowledge of the number of Located items and the combined use of the Offset Items and Maximum Items fields, a KMIP client is able to “page” or “browse” through a large Locate result set to find the items of interest.

A server is also free to implement limits on the number of items returned in a Locate if the client does not provide a Maximum Items value in order to prevent returning a complete list of items which may be much larger than a client expects when the set of attributes to match is not present or does not result in a reasonably small set of managed objects to return.

If operations are performed between Locate requests that alter the result set that is returned by a Locate operation, then items may be missed, or items may be returned more than once across separate Locate requests that simply vary the offset items value. The same circumstances apply when performing separate Locate requests without an offset value.

2.10 Key Life-cycle and Key State

[KMIP-SPEC] describes the key life-cycle model, based on the [SP800-57-1] key state definitions, supported by the KMIP protocol. Particular implications of the key life-cycle model in terms of defining time-related attributes of objects are discussed Using KMIP Functionality - Key State and Times.

3 Using KMIP Functionality

Text This section provides guidance on using the functionality described in the Key Management Interoperability Protocol (KMIP) Specification.

3.1 Authentication

As discussed in [\[KMIP-SPEC\]](#), a conforming KMIP implementation establishes and maintains channel confidentiality and integrity and provides assurance of server authenticity for KMIP messaging. Client authentication is performed according to the chosen KMIP authentication suite as specified in [\[KMIP-PROF\]](#). Other mechanisms for client and server authentication are possible and optional for KMIP implementations.

KMIP implementations that support the KMIP-defined Credential Types or use other vendor-specific mechanisms for authentication may use the optional Authentication structure specified inside the Request Header to include additional identification information. Depending on the server's configuration, the server may interpret the identity of the requestor from the Credential structure, contained in the Authentication structure if it is not provided during the channel-level authentication. For example, in addition to performing mutual authentication during a TLS handshake, the client passes the Credential structure (e.g., a username and password) in the request. If the requestor's username is not specified inside the client certificate and is instead specified in the Credential structure, the server interprets the identity of the requestor from the Credential structure. This supports use cases where channel-level authentication authenticates a machine or service that is used by multiple users of the KMIP server. If the client provides the username of the requestor in both the client certificate and the Credential structure, the server verifies that the usernames are the same. If they differ, the authentication fails, and the server returns an error. If no Credential structure is included in the request, the username of the requestor is expected to be provided inside the certificate. If no username is provided in the client certificate and no Credential structure is included in the request message, the server is expected to refuse authentication and return an error.

If authentication is unsuccessful, and it is possible to return an "authentication not successful" error, this error should be returned in preference to any other result status. This prevents status code probing by a client that is not able to authenticate.

Server decisions regarding which operations to reject if there is insufficiently strong authentication of the client are not specified in the protocol. However, see Section 3.2 for operations for which authentication and authorization are particularly important.

3.1.1 Credential

The Credential object defined in the [\[KMIP-SPEC\]](#) is a structure used to convey information about the client. The type of information conveyed within this object varies based on the type of credential (e.g. *Username and Password*, *Device Credential* and *Attestation*).

3.1.1.1 Username and Password Credential Type

[\[KMIP-SPEC\]](#) defines the Username and Password structure for the Credential Type Username and Password. The structure consists of two fields: Username and Password. Password is a recommended, but optional, field, which may be excluded only if the client is authenticated using one of the authentication suites defined in [\[KMIP-PROF\]](#). For example, if the client performs client certificate authentication during the TLS handshake, and the Authentication structure is provided in the Message Request, the Password field is an optional field in the Username and Password structure of the Credential structure.

The Credential structure is used to provide additional identification information. As described above, for certain use cases, channel-level authentication may only authenticate a machine or service that is used by multiple clients of the KMIP server. The Credential structure may be used in this scenario to identify individual clients by specifying the username in the Username and Password structure.

There is also a Hashed Password Credential Type that allows for authentication to be performed without a plaintext password (protected by the TLS transport) being sent from the client to the server.

3.1.2 Device Credential Type

The Device Credential may be used to uniquely identify back-end devices by specifying Device as the Credential Type in the Credential structure.

The Device Credential may be used in a proxy environment where the proxy authenticates with the client certificate and supports KMIP while the back-end devices may not support KMIP or TLS. An example is illustrated below:

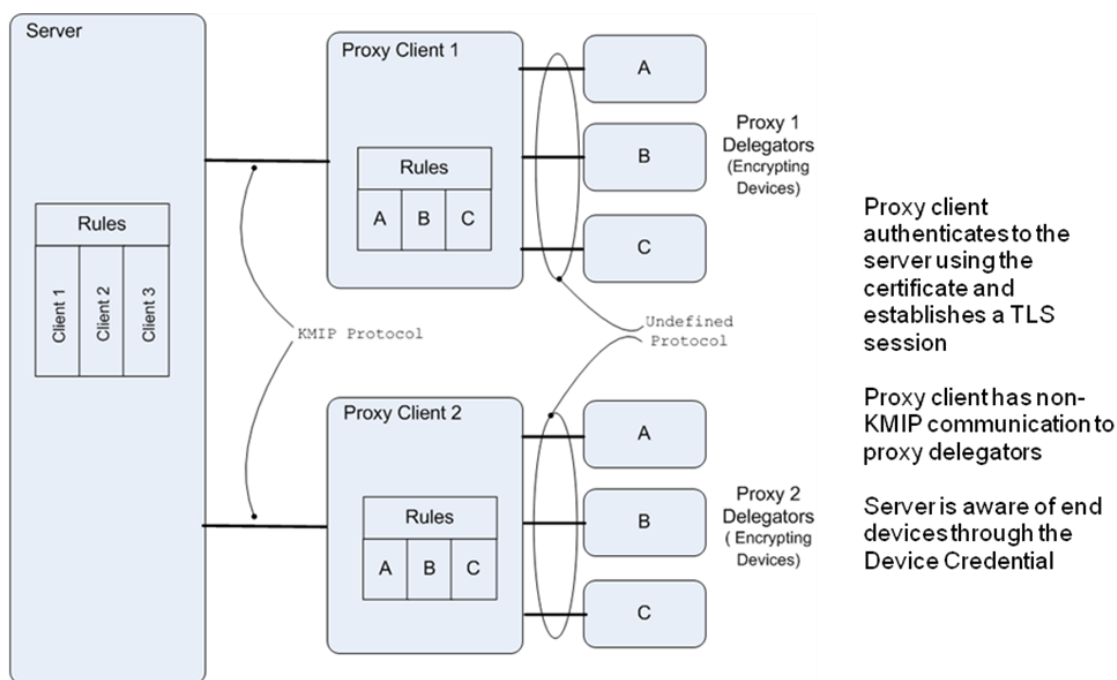


FIGURE 1: AGGREGATOR CLIENT EXAMPLE

The end device identifies itself with a device unique set of identifier values that include the device hardware serial number, the network identifier, the machine identifier, or the media identifier. For many of the self-encrypting devices there is a unique serial number assigned to the device during manufacturing. The ability to use network, machine, or media identifier explicitly should map to different device types and achieve better interoperability since different types of identifier values are explicitly enumerated. The device identifier is included for more generic usage. An optional password or shared secret may be used to further authenticate the device.

Server implementations may choose to enforce rules for uniqueness for different types of identifier values, combinations of TLS certificate used in combination with the Device Credential, and optionally enforce the use of a Device Credential password.

Four identifiers are optionally provided but are unique in aggregate:

1. Serial Number, for example the hardware serial number of the device
2. Network Identifier, for example the MAC address for Ethernet connected devices
3. Machine Identifier, for example the client aggregator identifier, such as a tape library aggregating tape drives
4. Media Identifier, for example the volume identifier used for a tape cartridge

The device identifier by choice of server policy may or may not be used in conjunction with the above identifiers to ensure uniqueness.

These additional identifiers are generally useful for auditing and monitoring encryption and could according to server policy be logged or used in server implementation specific validation.

A specific example for self-encrypting tape drive and tape library would be:

1. the tape drive has a serial number that is unique for that manufacturer and the vendor has procedures for maintaining and tracking serial number usage
2. a password optionally is created and stored either on the drive or the library to help authenticate the drive
3. the tape drives may be connected via fiber channel to the library and therefore have a World-Wide Name assigned
4. a machine identifier can be used to identify the tape library that is aggregating the device in question
5. the media identifier helps identify the individual media such as a tape cartridge for proof of encryption reporting

Another example using self-encrypting disk drives inside of a server would be:

1. the disk drive has a unique serial number
2. a password may be supplied by configuration of the drive or the server where the drive is located
3. the network identifier may come from the internal attachment identifier for the disk drive in the server
4. the machine identifier may come from a server's motherboard or service processor identifier,
5. and the media identifier comes from the volume name used by the server's operating system to identify the volume on the disk drive

Server implementations could control what devices may read and write keys and use the device credential fields to influence access control enforcement.

Another example applied to server virtualization and encryption built into virtualization would be:

1. the virtual machine instance has a unique identifier that is used for the serial number
2. the hypervisor supplies a shared secret that is used as the password to authenticate the virtual machine
3. the network identifier could be used to identify the MAC address of the physical server where the virtual machine is running
4. the machine identifier could be used to identify the hypervisor
5. the media identifier could be used to identify the storage volume used by the virtual machine

These are examples of usage and are not meant to define all device credential usage patterns nor restrict server specific implementations.

The device credentials may be explicitly added by the administrator or may be captured in line with the request and implicitly registered depending upon server policy.

When a server is not able to resolve the identifier values in the device credential to a unique client identification, it may choose to reject the request with an error code of operation failed and reason code of item not found.

3.2 Additional Authorization for Certain Operations

The authentication suite, as specified in [\[KMIP-PROF\]](#), describes how the client identity is established for KMIP-compliant implementations. This authentication is performed for all KMIP operations.

Certain operations that may be requested by a client via KMIP, particularly Revoke, Recover, Destroy, Archive, Import, Export and Obliterate, may have a significant impact on the availability of a key, on server performance and/or on key security. When a server receives a request for one of these operations, it should ensure that the client has authenticated its identity (see the Authentication Suites section in [\[KMIP-PROF\]](#)). The server should also ensure that the client requesting the operation is authorized to issue the request. It may also require additional authentication to ensure that the object owner or a security officer has issued that request. Even with such authentication and authorization, requests for these operations should be considered only a "hint" to the key management system, which may or may not choose to act upon this request depending on server policy.

3.3 Returning Related Objects

The key block returns a single object, with associated attributes and other data. For those cases in which multiple related objects are needed by a client, such as the private key and the related certificate, the client should issue multiple Get requests (potentially inside a single request using multiple batch items) to obtain these related objects.

3.4 Compromised Objects

A Cryptographic Object or Opaque Object may be compromised for a variety of reasons. In KMIP, a client indicates to the server that a Cryptographic Object is to be considered compromised by performing a Revoke Operation with a Revocation Reason of *Key Compromise* or *CA Compromise*. The KMIP client should provide a Compromise Occurrence Date (if the Revocation Reason is *Key Compromise* or *CA Compromise*) and if it is unable to estimate when the compromise occurred then it should provide a Compromise Occurrence Date equal to the Initial Date.

The KMIP specification [KMIP-SPEC] places no requirements on a KMIP server to perform any action on any Managed Object that references (i.e., via Link attributes) a Cryptographic Object or Opaque Object that a client has performed a Revoke operation with a Revocation Reason of *Key Compromise* or *CA Compromise*. However, KMIP users should be aware that there may be security relevant implications in continuing to use a Managed Cryptographic Object in the following circumstances:

- For a compromised Private Key, the linked Public Key and/or Certificate;
- For a compromised Public Key, the linked Private Key and/or Certificate;
- For a compromised Derived Key, the linked derived key and/or Secret Data Object

In these circumstances, it is the responsibility of the client to either check the state of the referenced Managed Object or to also perform a Revoke operation on the referenced Managed Object.

3.5 ID Placeholder

The ID Placeholder is obtained from the Unique Identifier returned in response to any operations that successfully completes and returns a Unique Identifier. The server SHALL copy the Unique Identifier into the ID Placeholder variable, where it is held until the completion of the operations remaining in the batched request or until a subsequent operation in the batch causes the ID Placeholder to be replaced. Subsequent operations in the batched request MAY make use of the ID Placeholder by using a Unique Identifier enumeration value of IDPlaceholder. For operations other than Locate which return more than one Unique Identifier (either as a Unique Identifier, Private Key Unique Identifier, or Public Key Unique Identifier), the first value in the response is the Unique Identifier value with respect to ID Placeholder handling.

Table 1 summarizes the ID Placeholder input and output values for KMIP Operations.

Operation	ID Placeholder at the beginning of the operation	ID Placeholder upon completion of the operation (in case of operation failure, a batch using the ID Placeholder stops)
Create		ID of new Object
Create Key Pair		ID of new Private Key (ID of new Public Key may be obtained via a Locate)
Create Split Key		ID of the split whose Key Part Identifier is 1
Join Split Key		ID of returned object
Register		ID of newly registered Object

Derive Key	Multiple Unique Identifiers may be specified in the request	ID of new Symmetric Key
Locate		ID of located Object
Get	ID of Object	no change
Validate		-
Get Attributes List/Modify/Add/Delete	ID of Object	no change
Activate	ID of Object	no change
Deactivate	ID of Object	no change
Revoke	ID of Object	no change
Destroy	ID of Object	no change
Obliterate	ID of Object	-
Archive/Recover	ID of Object	no change
Certify	ID of Public Key	ID of new Certificate
Re-certify	ID of Certificate	ID of new Certificate
Re-key	ID of Symmetric Key to be rekeyed	ID of new Symmetric Key
Re-key Key Pair	ID of Private Key to be rekeyed	ID of new Private Key (ID of new Public Key may be obtained via a Locate)
Obtain Lease	ID of Object	no change
Get Usage Allocation	ID of Key	no change
Check	ID of Object	no change

TABLE 1: ID PLACEHOLDER PRIOR TO AND RESULTING FROM A KMIP OPERATION

3.6 Use of Meta-Data Only (MDO) Keys

Meta-Data Only (MDO) keys are those Managed Key Objects for which no Key Value is present. This may be a result of the KMIP client only wanting to register information (Meta-Data) about the key with a Key Management System, without having the key itself leave the client's physical boundary. One such example could be for keys created and stored within a Hardware Security Module (HSM), with a policy that does not allow for the keys to leave its hardware. In such cases, the KMIP client will not include a Key Value within the Key Block during a Register operation, although it may optionally include a Key Value Location attribute indicating the location of the Key Value instead. For such keys, as part of the Register operation, the server will create a Key Value Present attribute and set it to false to indicate the key value is not stored on the server.

The KMIP protocol does not support the addition of a Key Value to an existing MDO key object on the server. If for some reason the client wanted to do this, it would have to carry out another Register operation and create a new managed object with the Key Value.

3.7 Using the Same Asymmetric Key Pair in Multiple Algorithms

There are mathematical relationships between certain asymmetric cryptographic algorithms such as the Digital Signature Algorithm (DSA) and Diffie-Hellman (DH) and their elliptic curve equivalents ECDSA and ECDH that allow the same asymmetric key pair to be used in both algorithms. In addition, there are overlaps in the key format used to represent the asymmetric key pair for each algorithm type.

Even though a single key pair may be used in multiple algorithms, the KMIP Specification has chosen to specify separate key formats for representing the asymmetric key pair for use in each algorithm. This approach keeps KMIP in line with the reference standards (e.g., NIST [FIPS 186-4], ANSI [X9.42], etc.) from which the key formats are obtained and the best practice documents (e.g., NIST [SP800-57-1], NIST [SP800-56A] etc.) which recommend that a key pair only be used for one purpose.

3.8 Non-Cryptographic Objects

The KMIP protocol allows clients to register Secret Data objects. Secret Data objects may include passwords or data that are used to derive keys.

KMIP defines Secret Data as cryptographic objects. Even if the object may not be used for cryptographic purposes, clients may still set certain attributes, such as the Cryptographic Usage Mask, for this object unless otherwise stated. Similarly, servers set certain attributes for this object, including the Digest, State, and certain Date attributes, even if the attributes may seem relevant only for other types of cryptographic objects.

When registering a Secret Data object, the following attributes are set by the server:

- Unique Identifier
- Object Type
- Digest
- State
- Initial Date
- Last Change Date

When registering a Secret Data object for non-cryptographic purposes, the following attributes are set by either the client or the server:

- Cryptographic Usage Mask

3.9 Split Key

KMIP allows a client to register a Split Key that was created or otherwise obtained by the client. To request that the server generate a split key, the client sends a Create Split Key request that includes the Split Key parameters (Split Key Parts, Split Key Threshold, Split Key Method) and desired key attributes (e.g. Object Type, Cryptographic Length). If the client supplies the Unique Identifier of an existing base key in a Create Split Key request, the server will use the supplied key in the key splitting operation instead of generating a new one. The server will respond with a list of Unique Identifiers for the newly created Split Keys.

The client may want to add link attributes to more easily locate the complete set of related Split Keys. The client adds a Previous Link from the Split Key with Key Part Identifier K to the Split Key with Key Part Identifier K-1 and a Next Link to the Split Key with Key Part Identifier K+1. Denoting the value of Split Key Parts by N, the client adds a Previous Link from the Split Key with Key Part Identifier 1 to the Split Key with Key Part Identifier N and a Next Link from the Split Key with Key Part Identifier N to the Split Key with Key Part Identifier 1. If the client supplies the Unique Identifier of an existing base key in a Create Split Key request, the client may want to add a Parent Link attribute from each newly generated Split Key to the base key that was supplied in the Create Split Key request.

To request that the server recombine a set of split keys, the client sends a Join Split Key request that includes the type of object to be returned (e.g. Symmetric Key, Private Key, or Secret Data) and a list of Unique Identifiers of the Split Keys to be combined. The number of Unique Identifiers in the request

needs to be at least the value of Split Key Threshold in the Split Keys to ensure the server will be able to combine the keys according to the Split Key Method. The server will respond with the Unique Identifier of the key obtained by combining the provided Split Keys.

For PolynomialSharingGF2_8 based splits, the specific polynomial being used varies between implementations. The OASIS Security Algorithms and Methods (SAM) TC has defined two algorithms in the *SAM Threshold Sharing Schemes Version 1.0* [SAM-TSS] which are referred to in KMIP as Polynomial-283 (Polynomial 1 within SAM-TSS) and Polynomial-285 (Polynomial 2 within SAM-TSS).

Implementations should explicitly specify the SplitKeyPolynomial in KMIP operations in order to achieve interoperability as different KMIP implementations have historically defaulted to different polynomials. The Split Key Algorithms section of the [KMIP-SPEC] for PolynomialSharingGF2_8 has been replaced with a reference to [SAM-TSS].

3.10 Asymmetric Concepts with Symmetric Keys

The Cryptographic Usage Mask attribute is intended to support asymmetric concepts using symmetric keys. This is common practice in established crypto systems: the MAC is an example of an operation where a single symmetric key is used at both ends, but policy dictates that one end may only generate cryptographic tokens using this key (the MAC) and the other end may only verify tokens. The security of the system fails if the verifying end can use the key to perform generation operations.

In these cases, it is not sufficient to describe the usage policy on the keys in terms of cryptographic primitives like “encrypt” vs. “decrypt” or “sign” vs. “verify”. There are two reasons why this is the case.

- In some of these operations, such as MAC generation and verification, the same cryptographic primitive is used in both of the complementary operations. MAC generation involves computing and returning the MAC, while MAC verification involves computing that same MAC and comparing it to a supplied value to determine if they are the same. Thus, both generation and verification use the “encrypt” operation, and the two usages are not able to be distinguished by considering only “encrypt” vs. “decrypt”.
- Some operations which require separate key types use the same fundamental cryptographic primitives. For example, encryption of data, encryption of a key, and computation of a MAC all use the fundamental operation “encrypt”, but in many applications, securely differentiated keys are used for these three operations. Simply looking for an attribute that permits “encrypt” is not sufficient.

Allowing the use of these keys outside of their specialized purposes may compromise security. Instead, specialized application-level permissions are necessary to control the use of these keys. KMIP provides several pairs of such permissions in the Cryptographic Usage Mask, such as:

MAC GENERATE MAC VERIFY	For cryptographic MAC operations. Although it is possible to compose certain MACs using a series of encrypt calls, the security of the MAC relies on the operation being atomic and specific.
GENERATE CRYPTOGRAM VALIDATE CRYPTOGRAM	For composite cryptogram operations such as financial CVC or ARQC. To specify exactly which cryptogram the key is used for it is also necessary to specify a <i>role</i> for the key
TRANSLATE ENCRYPT TRANSLATE DECRYPT TRANSLATE WRAP TRANSLATE UNWRAP	To accommodate secure routing of traffic and data. In many areas that rely on symmetric techniques (notably, but not exclusively financial networks), information is sent from place to place encrypted using shared symmetric keys. When encryption keys are changed, it is desirable for the change to be an atomic operation, otherwise distinct unwrap-wrap or decrypt-encrypt steps risk

	leaking the plaintext data during the translation process. <i>TRANSLATE ENCRYPT/DECRYPT</i> is used for data encipherment. <i>TRANSLATE WRAP/UNWRAP</i> is used for key wrapping.
--	---

TABLE 2: CRYPTOGRAPHIC USAGE MASKS PAIRS

3.11 Key Rotation

Prior versions of KMIP supported client-side key rotation methods. This version of KMIP adds support for automatic rotation. A client may now set the RotateAutomatic attribute to true to enable automatic rotation of keys by the server.

Additional attributes (RotateGeneration and RotateLatest) have been added to KMIP to enable more efficient handling and tracking of a series of keys that have been rotated.

3.12 Key Block

The protocol uses the Key Block structure to transport a key to the client or server. This Key Block consists of the Key Value Type, the Key Value, and the Key Wrapping Data. The Key Value Type identifies the format of the Key Material, e.g., Raw format or Transparent Key structure. The Key Value consists of the Key Material and optional attributes. The Key Wrapping Data provides information about the wrapping key and the wrapping mechanism and is returned only if the client requests the Key Value to be wrapped by specifying the Key Wrapping Specification inside the Get Request Payload. The Key Wrapping Data may also be included inside the Key Block if the client registers a wrapped key.

The protocol allows any attribute to be included inside the Key Value and allows these attributes to be cryptographically bound to the Key Material (i.e., by signing, MACing, encrypting, or both encrypting and signing/MACing the Key Value). Some of the attributes that may be included include the following:

- Unique Identifier – uniquely identifies the key
- Cryptographic Algorithm (e.g., AES, 3DES, RSA) – this attribute is either specified inside the Key Block structure or the Key Value structure
- Cryptographic Length (e.g., 128, 256, 2048) – this attribute is either specified inside the Key Block structure or the Key Value structure
- Cryptographic Usage Mask– identifies the cryptographic usage of the key (e.g., Encrypt, Wrap Key, Export)
- Cryptographic Parameters – provides additional parameters for determining how the key may be used
 - Block Cipher Mode (e.g., CBC, NISTKeyWrap, GCM) – this parameter identifies the mode of operation, including block cipher-based MACs or wrapping mechanisms
 - Padding Method (e.g., OAEP, X9.31, PSS) – identifies the padding method and if applicable the signature or encryption scheme
 - Hashing Algorithm (e.g., SHA-256) – identifies the hash algorithm to be used with the signature/encryption mechanism or Mask Generation Function; note that the different HMACs are defined individually as algorithms and do not require the Hashing Algorithm parameter to be set
 - Key Role Type – Identifies the functional key role (e.g., DEK, KEK)
- State (e.g., Active)
- Dates (e.g., Activation Date, Process Start Date, Protect Stop Date)

- Custom Attribute – allows vendors and clients to define vendor-specific attributes; may also be used to prevent replay attacks by setting a nonce

3.13 Key Encoding

Two parties receiving the same key as a Key Value Byte String make use of the key in exactly the same way in order to interoperate. To ensure that, it is necessary to define a correspondence between the abstract syntax of Key and the notation in the standard algorithm description that defines how the key is used. The next sections establish that correspondence for the algorithms AES [FIPS 197] and Triple-DES [SP800-67].

3.13.1 AES Key Encoding

AES Key Encoding [FIPS 197] section 5.2, titled Key Expansion, uses the input key as an array of bytes indexed starting at 0. The first byte of the Key becomes the key byte in AES that is labeled index 0 in [FIPS 197] and the other key bytes follow in index order.

Proper parsing and key load of the contents of the Key for AES is determined by using the following Key byte string to generate and match the key expansion test vectors in [FIPS 197] Appendix A for the 128-bit (16 byte) AES Cipher Key: 2B 7E 15 16 28 AE D2 A6 AB F7 15 88 09 CF 4F 3C.

3.13.2 Triple-DES Key Encoding

A Triple-DES key consists of three keys for the cryptographic engine (Key1, Key2, and Key3) that are each 64 bits (even though only 56 are used); the three keys are also referred to as a key bundle (KEY) [SP800-67]. A key bundle may employ either two or three mutually independent keys. When only two are employed (called two-key Triple-DES), then Key1 = Key3.

Each key in a Triple-DES key bundle is expanded into a key schedule according to a procedure defined in [SP800-67] Appendix A. That procedure numbers the bits in the key from 1 to 64, with number 1 being the left most, or most significant bit. The first byte of the Key is bits 1 through 8 of Key1, with bit 1 being the most significant bit. The second byte of the Key is bits 9 through 16 of Key1, and so forth, so that the last byte of the KEY is bits 57 through 64 of Key3 (or Key2 for two-key Triple-DES).

Proper parsing and key load of the contents of Key for Triple-DES is determined by using the following Key byte string to generate and match the key expansion test vectors in [SP800-67] Appendix B for the key bundle:

Key1 = 0123456789ABCDEF

Key2 = 23456789ABCDEF01

Key3 = 456789ABCDEF0123

3.13.3 AES-XTS Key Encoding

AES-XTS¹ is an implementation of the AES encryption algorithm with the XTS (XEX (XOR Encrypt XOR) Tweakable Block Cipher with Ciphertext Stealing) cipher mode (see [SP800-38E]). What is unique about the XTS mode is that it requires the use of two AES keys. AES-XTS 512 requires two 256-bit AES keys while AES-XTS 256 requires two 128-bit AES keys.

KMIP implementations should treat the required XTS keys as two distinct keys (e.g. two 256-bit AES keys). KMIP operations (e.g. Create, Register, Get) associated with these two keys are batched within the profile to logically keep the handling of these associated keys together. A simple custom attribute (X-ID) can be used to specify a naming convention that associates the two AES keys. Attributes Next Link and Previous Link are used to associate the two AES keys with one another on the KMIP server.

¹ Also referred to as XTS-AES

3.14 Attributes

All managed objects within KMIP have associated attributes. Initial attribute values are provided during object creation or.

Attributes have associated rules. They may be mandatory or optional. They may be single instance, or multi-instance. They may be set or updated only by the client or by the server. Within the Specification, each attribute section contains a table defining these rules.

Specific operations allow for working with attributes.

- Attribute values may be created using either Add Attribute or Set Attribute
- Attribute values may be modified using either Modify Attribute or Set Attribute.
- Set Attribute is more appropriate for single-instance attributes as it does not require knowledge of whether the attribute previously had a value. Set Attribute cannot be used for multi-instance attributes.
- Both multi-instance and single-instance attributes may be removed using Delete Attributes.
- Instances of multi-instance attributes are identified by specifying their current value when using either Modify Attribute or Delete Attribute.
- Adjust Attribute should be used where applicable, both because it is more convenient and is transaction-safe.

3.15 Key State and Times

[KMIP-SPEC] provides a number of time-related attributes, including the following:

- Original Creation Date: The date and time when the managed object was originally created, which can be different from when the object is registered with a key management server
- Rotate Date: The date and time when the managed cryptographic object was last rotated by the server (using server-side automatic rotation)
- Initial Date: The date and time when the managed cryptographic object was first created by or registered at the server.
- Activation Date: The date and time when the managed cryptographic object should begin to be used for applying cryptographic protection to data.
- Process Start Date: The date and time when a managed symmetric key object should begin to be used for processing cryptographically protected data. The managed symmetric key object should not be used prior to this date.
- Protect Stop Date: The date and time when a managed symmetric key object should no longer be used for applying cryptographic protection to data
- Deactivation Date: The date and time when the managed cryptographic object should no longer be used for applying cryptographic protection (e.g., encryption, signing, wrapping, MACing, deriving). Under extraordinary circumstances and when special permission is granted the managed symmetric key object can be used for decryption, signature verification, unwrapping, or MAC verification,
- Destroy Date: The date and time when the managed cryptographic object was destroyed
- Compromise Occurrence Date: The date and time when the managed cryptographic object was first believed to be compromised.
- Compromise Date: The date and time when the managed cryptographic object entered the compromised state.
- Archive Date: The date and time when the managed object was placed in Off-Line storage.

These attributes apply to all objects. However, certain attributes (such as the Initial Date) are not specified by the client and are implicitly set by the server.

In using these attributes, the following guidelines should be observed:

- As discussed for each of these attributes in [\[KMIP-SPEC\]](#), a number of these times are set once, and it is not possible for the client or server to modify them. However, several of the time attributes (particularly the Activation Date, Protect Start Date, Process Stop Date and Deactivation Date) may be set by the server and/or requested by the client. Coordination of time-related attributes between client and server, therefore, is primarily the responsibility of the server, as it manages the cryptographic object and its state. However, special conditions related to time-related attributes, governing when the server accepts client modifications to time-related attributes, may be communicated out-of-band between the client and server outside the scope of KMIP.
- In general, state transitions occur as a result of operational requests. However, clients may need to specify times in the future for such things as Activation Date, Deactivation Date, Process Start Date, and Protect Stop Date. KMIP also allows clients to specify times in the past for such attributes as Activation Date and Deactivation Date.
- It is valid to have a projected Deactivation Date when there is no Activation Date. This means, however, that the key is not yet active, even though its projected Deactivation Date has been specified. A valid Deactivation Date is greater than or equal to the Activation Date (if the Activation Date has been set).
- The Protect Stop Date may be equal to but may not be later than the Deactivation Date. Similarly, the Process Start Date may be equal to, but may not precede, the Activation Date. KMIP implementations should consider specifying both these attributes, particularly for symmetric keys, as a key may be needed for processing protected data (e.g., decryption) long after it is no longer appropriate to use it for applying cryptographic protection to data (e.g., encryption).
- KMIP does not allow an Active object to be destroyed with the Destroy operation. The server returns an error if the client invokes the Destroy operation on an Active object. To destroy an Active object, clients first call the Deactivate or Revoke operation or explicitly set the Deactivation Date of the object. Once the object is in Deactivated state, clients may destroy the object by calling the Destroy operation. These operations may be performed in a batch.
- After a cryptographic object is destroyed, a key management server may retain certain information about the object, such as the Unique Identifier and other attributes.

3.16 Application Specific Information

The Application Specific Information attribute is used to store data which is specific to the application(s) using the object. Some examples of Application Namespace and Application Data pairs are given below.

- SMIME, 'someuser@company.com'
- TLS, 'some.domain.name'
- Volume Identification, '123343434'
- File Name, 'secret.doc'
- Client Generated Key ID, '450994003'

The following Application Namespaces are recommended:

- SMIME
- TLS
- IPSEC

- HTTPS
- PGP
- Volume Identification
- File Name
- LTO4, LTO5, and LTO6
- LIBRARY-LTO, LIBRARY-LTO4, LIBRARY-LTO5 and LIBRARY-LTO6

KMIP provides optional support for server-generated Application Data. Clients may request the server to generate the Application Data for the client by omitting Application Data while setting or modifying the Application Specific Information attribute. A server only generates the Application Data if the Application Data is completely omitted from the request, and the client-specified Application Namespace is recognized and supported by the server. An example for requesting the server to generate the Application Data is shown below:

```
AddAttribute(Unique ID, AppSpecInfo{AppNameSpace='LIBRARY-LTO4'});
```

If the server does not recognize the namespace, the “Application Namespace Not Supported” error is returned to the client.

If the Application Data is provided, and the Application Namespace is recognized by the server, the server uses the provided Application Data, and does not generate the Application Data for the client. In the example below, the server stores the Application Specific Information attribute with the Application Data value set to null.

```
AddAttribute(Unique ID, AppSpecInfo{AppNameSpace='LIBRARY-LTO4', AppData=null});
```

3.17 Description and Comment Attributes

The Description and Comment attributes are used to convey information about the purpose and use of an object. Description is intended to be a concise imperative statement about an object (e.g. “Root Key for internal servers”). Comment is a complementary field which allows more verbose communication regarding what activity or why an activity applies to an object (e.g. “Make sure to update new internal servers with the Root key as they are brought on line”). Both attributes are optional, and their contents should be used for informational purposes and not for policy enforcement.

3.18 Cryptographic Length of Asymmetric Keys

The value (e.g., 2048 bits) referred to in the KMIP *Cryptographic Length* attribute for an asymmetric (public or private) key may be misleading, since this length only refers to certain portions of the mathematical values that comprise the key. The actual length of all the mathematical values comprising the public or the private key is longer than the referenced value. This is true for both classic and PQC asymmetric algorithms.

This point may be illustrated by looking at the components of a RSA public and private key (a classic asymmetric algorithm). The RSA public key is comprised of a modulus (n) and an (public) exponent (e). When one indicates that the RSA public key is 2048 bits in length that is a reference to the bit length of the modulus (n) only. So, the full length of the RSA public key is actually longer than 2048 bits, since it also includes the length of the exponent (e) and the overhead of the encoding (e.g., ASN.1) of the key material.

The RSA private key is comprised of a modulus (n), the public exponent (e), the private exponent (d), prime 1 (p), prime 2 (q), exponent 1 (d mod (p-1)), exponent 2 (d mod (q-1)), and coefficient ((inverse of q) mod p). Once again, the 2048-bit key length is referring only to the length of the modulus (n), so the overall length of the private key would be longer given the number of additional components which comprise the key and the overhead of encoding (e.g., ASN.1) of the key material.

KMIP implementations need to ensure they do not make assumptions about the actual length of asymmetric (public and private) key material based on the value specified in the *Cryptographic Length* attribute.

3.19 Constraints

Just as Defaults can now be supplied to or by the server, the server may be constrained to only permit certain cryptographic parameters to be used. For example, one may constrain the server to only allow public keys with a particular `CryptographicAlgorithm` (e.g., ECDSA) and/or a particular `CryptographicLength` (e.g., 521). Such Constraints are expressed in a Constraints object with `ObjectTypes`, `ObjectGroups` and `Attributes`. The client may obtain a list of constraints from a server via `GetConstraints`, and sufficiently privileged clients can set the constraints via `SetConstraints`.

3.20 Multi-instance Hash

The `Digest` attribute contains the output of hashing a managed object, such as a key or a certificate. The server always generates the SHA-256 hash value when the object is created or generated. KMIP allows multiple instances of the digest attribute to be associated with the same managed object. For example, it is common practice for publicly trusted CAs to publish two digests (often referred to as the fingerprint or the thumbprint) of their certificate: one calculated using the SHA-1 algorithm and another using the MD5 algorithm. In this case, each digest would be calculated by the server using a different hash algorithm.

3.21 Extractable and Sensitive Attributes

KMIP supports attributes which can be used to indicate the sensitivity of a cryptographic object and whether a cryptographic object may leave the server. These attributes are modeled after similar attributes within [\[PKCS#11\]](#).

The *Sensitive* attribute is used to specify whether a cryptographic object can only leave the KMIP server (via the `Get` operation) in wrapped (encrypted) form.

The *Always Sensitive* attribute is used to track whether a cryptographic object has always been considered sensitive during its existence within the KMIP server.

The *Extractable* attribute is used to control whether a cryptographic object is ever allowed to leave the KMIP server.

The *Never Extractable* attribute is used to track whether extraction of a cryptographic object has always been restricted during its existence within the KMIP server.

3.22 Key Format Type

A client may wish to use the digest attribute of an object to compare objects. If a client does not specify a key format type on a constructor, the server may choose any of a number of key format types to use to encode that object. If the server chooses a format that the client cannot handle, then the client will be unable to use the digest of that format in any meaningful way. The key format attribute was added so that the client can tell the server exactly which format to use and if that format is not supported then fail the construction. The server is free to add digests of other formats in addition to the format specified by the client.

3.23 Using the “Raw” Key Format Type

The “raw” key format is intended to be used for “a key that contains only cryptographic key material, encoded as a string of bytes”. The “raw” key format supports situations such as “non-KMIP-aware end-clients are aware how wrapped cryptographic objects (possibly Raw keys) from the KMIP server should be used without having to rely on the attributes provided by the `Get Attributes` operation” and in that regard is similar to the `Opaque` key format type.

3.24 PKCS#12 Key Format

PKCS#12 was originally introduced in the early days of PKI and is used as a keystore format and as a key and certificate exchange mechanism. The addition of PKCS#12 to KMIP as a key format type is intended as a means of admitting the basic forms of PKCS#12 into a managed environment, as well as extracting basic forms of PKCS#12 from the KMIP world to be consumed/used in existing environments and workflows.

The intent of KMIP's PKCS#12 key format is that a PKCS#12-conformant blob can be Registered by the client, and a PKCS#12-conformant blob can be returned from a KMIP server on a Get. It is presumed that a single private key is what is being Registered (along with its related certificate or certificate chain), so the Unique Identifier that KMIP servers return on a Register PKCS#12 operation is that of the private key. It is also presumed that the client doing a Get with a key format type of PKCS#12 will be passing in the private key's Unique Identifier, and that the KMIP server is responsible for returning a PKCS#12 blob with the referenced private key and any related certificates.

If the PKCS#12 blob being Registered has multiple private keys, then the client can indicate which ONE of them is being Registered on a particular request by specifying the PKCS#12 Friendly Name attribute. If the client does not supply the Friendly Name (a.k.a. "alias"), then the server is free to just Register the first that it finds in the PKCS#12 blob. If the client wants more than one private key within a PKCS#12 blob to be Registered from a single PKCS#12 blob, then it will have to request repeatedly with that PKCS#12 blob using a different Friendly Name for each request.

Those familiar with PKCS#12 objects, particularly those containing sensitive material such as private keys, know that such objects are usually password-protected. KMIP honors such constraints by requiring SecretData objects to be referenced and used for the normal password-based encryption (PBE) schemes employed to protect PKCS#12 objects. Hence Register requests for objects with PKCS#12 key format must supply a Link attribute with a LinkType of "PKCS#12 Password Link", so that the server can find the SecretData object containing the password for the PKCS#12 blob being sent to the server and decrypt and verify its contents. Such a SecretData object must be in the Active state and have a Cryptographic Usage Mask that indicates "DeriveKey" for it to be used for password-based key derivation. At the end of the Register operation, one can expect the KMIP server to have decrypted and verified the contents of the PKCS#12 blob, registered the (specified or defaulted) private key, added the PKCS#12 Password Link to the SecretData in the new PrivateKey, found and Registered the certificate(s) that make up the private key's certificate chain, and linked these Certificates from the PrivateKey via the PKCS#12 CertificateLink (the link to the first Certificate is from the PrivateKey; any subsequent Certificates would be linked by a PKCS#12 Certificate Link in the previous Certificate).

When a client issues a Get PKCS#12 key format request on a PrivateKey Unique Identifier, the KMIP server traverses all the links set up on the Register to reassemble the PKCS#12 blob and protect it with the SecretData specified on the PrivateKey PKCS#12 Password Link. If the client wants a different password on the PKCS#12 blob resulting from a Get, it should modify the PKCS#12 Password Link on the PrivateKey to a different SecretData before performing the Get. Similarly, if the client wants a different Friendly Name for the key entry in a PKCS#12 blob, it should modify the PKCS#12 Friendly Name attribute in the PrivateKey object before performing the Get.

A client can set up all this machinery by hand to be able to Get PKCS#12 key format without a corresponding Register PKCS#12 key format having been used – i.e. you can request returning a PKCS#12 blob for existing managed objects independent of how those objects were created or registered.

3.25 Protection Storage Mask

The server may have multiple ways in which keys can be stored and protected. The client may wish to avail themselves of particular variations. The client can specify an ordered list of preferences for said storage, one of which the server must meet in its entirety or fail the request. One example of this might be the requirement for cryptographic material to remain within a geographical location or boundary (data sovereignty)

3.26 Unique Identifiers

For clients that require unique identifiers in a special form, out-of-band registration/configuration may be used to communicate this requirement to the server. The use of a Universally Unique Identifier (UUID) [RFC 9562] is recommended – but servers are not required to select this as a format, so clients must be willing to handle responses that are in different formats.

3.27 Usage Allocation

Usage should be allocated and handled carefully at the client, since power outages or other types of client failures (crashes) may render allocated usage lost. For example, in the case of a key being used for the encryption of tapes, such a loss of the usage allocation information following a client failure during encryption may result in the necessity for the entire tape backup session to be re-encrypted using a different key, if the server is not able to allocate more usage. It is possible to address this through such approaches as caching usage allocation information on stable storage at the client, and/or having conservative allocation policies at the server (e.g., by keeping the maximum possible usage allocation per client request moderate). In general, usage allocations should be as small as possible; it is preferable to use multiple smaller allocation requests rather than a single larger request to minimize the likelihood of unused allocation.

3.28 Canceling Asynchronous Operations

If an asynchronous operation is cancelled by the client, no information is returned by the server in the result code regarding any operations that may have been partially completed. Identification and remediation of partially completed operations is the responsibility of the server.

It is the responsibility of the server to determine when to discard the status of asynchronous operations. The determination of how long a server should retain the status of an asynchronous operation is implementation-dependent and not defined by KMIP.

Once a client has received the status on an asynchronous operation other than “pending”, any subsequent request for status of that operation may return either the same status as in a previous polling request or an “unavailable” response.

3.29 Full Async

“Mandatory” Asynchronous operations in the request header establishes that the server will return Asynchronous Correlation Value for all the batch items. The expectation is that one of the batch items may have a longer response time than a client is willing to maintain a connection. This is useful in an IoT environment that has limited resources.

3.30 Querying Outstanding Asynchronous Requests

With the growing usage of asynchronous services in KMIP, the inability of clients to see what requests are still pending on the server may become troublesome for clients and administrators were it not for the advent of QueryAsynchronousRequests (a separate operation, rather than another parameter of the Query operation). This operation permits the client to request the status of ALL outstanding requests (meaning the results have not been returned via the normal Poll or Cancel mechanisms) or a subset of outstanding asynchronous requests, either by Operation type(s) or by particular Asynchronous Correlation Value(s) or both.

3.31 Process

There may be occasions where an asynchronous service was initially requested, but the requestor now requires an immediate response. One possible use case might be a client who requested a new KeyPair to be created asynchronously, but for which a compromise is reported in the interim and thus the new KeyPair needs to be created **now**. The Process operation is intended for this usage. It takes an Asynchronous Correlation Value (or multiple), and the server will not return results from the Process request to the client until the server has ensured that the next Poll by the client for that/those Asynchronous Correlation Value(s) will return a status other than “Operation Pending”. The client view of this request is that it moves the original request(s) from “do it sometime” to “do it now”, Server implementations may choose to increase service priority for these requests or other actions, as long as the semantic is that the next Poll will report a status other than “Operation Pending”. The server may report an error or normal completion, just not “Operation Pending”.

3.32 Cryptographic Services

KMIP supports creation and registration of managed objects and retrieval of managed objects in both plaintext and optionally wrapped with another managed object. KMIP also includes support for a subset of the operations necessary for certificate management (certifying certificate requests and validating certificate hierarchies). KMIP defines a range of Hash-based and MAC-based key derivation options.

There are certain situations in which having capability for a KMIP client to request cryptographic operations from a KMIP server is beneficial in terms of simplifying the client implementation, strengthening the integration between the key management and cryptographic operations, or improving the overall security of a solution.

KMIP includes support for cryptographic services in the form of client-to-server operations for cryptographic services using managed objects for encryption, decryption, signature generation, signature verification, MAC generation, MAC verification, random number generation, and general hashing.

This support for cryptographic services is similar to the approach taken in KMIP for certificates. The protocol supports a base set of operations on certificates that enable a key manager to act as a proxy for a Certification Authority (CA) or in fact operate as a CA in the contexts where that is appropriate. A KMIP server supporting cryptographic services may be acting as a proxy for another cryptographic device or in fact operating as a cryptographic device in the contexts where that is appropriate.

Similarly, server policy regarding accepting random data from a client should reflect the level of confidence that that server has in a particular client or all clients. Issues in the quality or integrity of random data provided in RNG Seed can affect key creation, nonce and IV generation, client-server TLS session key creation, and the random data delivered to clients with the RNG Retrieve Operation. KMIP, as a protocol, does not enforce restrictions on the quality or nature of the random data provided by a client in the RNG Seed operation.

A KMIP server that supports the RNG Retrieve and RNG Seed operations may have a single RNG for the server, an RNG which is shared in an unspecified manner by KMIP clients or a separate RNG for each KMIP client. There is no requirement for the KMIP server to implement any specific RNG model.

3.32.1 Streaming Cryptographic Services

KMIP allows for multi-part (streaming) operations where the data required for the operations may be provided in multiple separate requests. Each of the existing encryption, decryption, signature generation, signature verification, MAC generation, MAC verification, and general hashing operations now support additional parameters to allow for each of the stages of multi-part usage – initialization, update, and finalization.

The server returns to the client a Correlation Value in the first response. This correlation value is used by the client in each subsequent request for performing multi-part (streaming) operations. Each cryptographic operation has additional optional parameters in the request that enable the client to indicate which stage of multi-part usage (initialization, update, and finalization) is being requested.

A client can use the Query Capability Information to determine if a KMIP server supports the multi-part (streaming) operations by checking if the Streaming Capability is set to true in the response to a Query operation with the Query Function value of Query Capability Information.

3.32.2 Security Considerations for Server State Handling.

When a server is processing a correlation value either for asynchronous or multi-part (streaming) operations and matching an incoming request against server state on behalf of a client, the following may be relevant to consider as part of determining whether or not to accept the specified correlation value:

- Matching TLS mutual authentication client credentials
- Matching KMIP authentication header information
- Matching link-level (network end-point) details (i.e. source address information)
- Matching TLS session identifiers
- Requiring the first and last operations to be within a specified time period

- Requiring each operation to be within a specified time period
- Re-check the access rights on all referenced managed objects

The KMIP specification does not require that any of these items are considered as the determination of which items are relevant is both server implementation and security context dependent.

3.33 Archive Operations

When the Archive operation is performed, it is recommended that a unique identifier and a minimal set of attributes be retained within the server for operational efficiency. In such a case, the retained attributes may include Unique Identifier and State.

3.34 Certify and Re-certify

The key management system may contain multiple embedded CAs or may have access to multiple external CAs. How the server routes a certificate request to a CA is vendor-specific and outside the scope of KMIP. A client may specify a preference for the CA used for signing a Certificate Request by including the X.509 Certificate Issuer attribute in the Certify or Re-certify request, but whether the server chooses to act on this preference is vendor-specific and outside the scope of KMIP.

[KMIP-SPEC] supports multiple options for submitting a certificate request to the key management server within a Certify or Re-Certify operation. It is a vendor decision as to whether the key management server offers certification authority (CA) functionality or proxies the certificate request onto a separate CA for processing. The type of certificate request formats supported is also a vendor decision, and this may, in part, be based upon the request formats supported by any CA to which the server proxies the certificate requests.

All certificate request formats for requesting X.509 certificates specified in [KMIP-SPEC] (i.e., PKCS#10, PEM and CRMF) provide a means for allowing the CA to verify that the client that created the certificate request possesses the private key corresponding to the public key in the certificate request. This is referred to as Proof-of-Possession (POP). However, it should be noted that in the case of the CRMF format, some CAs may not support the CRMF POP option, but instead rely upon the underlying certificate management protocols (i.e., CMP and CMC) to provide POP. In the case where the CA does not support POP via the CRMF format (including CA functionality within the key management server), an alternative certificate request format (i.e., PKCS#10, PEM) would need to be used if POP needs to be verified.

3.35 Using Offset in Re-key and Re-certify Operations

The Re-key, Re-key Key Pair, and Re-certify operations allow the specification of an offset interval.

The Re-key and the Re-key Key Pair operations allow the client to specify an offset interval for activation of the key. This offset specifies the duration of time between the time the request is made and the time when the activation of the key occurs. If an offset is specified, all other times for the new key are determined from the new Activation Date, based on the intervals used by the previous key, i.e., from the Activation Date to the Process Start Date, Protect Stop Date, etc.

The Re-certify operation allows the client to specify an offset interval that indicates the difference between the Initial Date of the new certificate and the Activation Date of the new certificate. As with the Re-key operation, all other times for the certificate are determined using the intervals used for the previous certificate.

Note that in the Re-key and Re-key Key Pair operations if activation date, process start date, protect stop date and deactivation date are obtained from the existing key, and the initial date is obtained from the current time, then the deactivation/activation date/process start date/protect stop date is smaller or less than initial date. KMIP allows forward-dating of these values to prevent a contradiction.

3.36 Certificate Renewal, Update, and Re-key

The process of generating a new certificate to replace an existing certificate may be referred to by multiple terms, based upon what data within the certificate is changed when the new certificate is created. In all situations, the new certificate includes a new serial number and new validity dates.

[KMIP-SPEC] uses the following terminology which is aligned with the definitions found in IETF [RFC3647] and [RFC4949]:

- *Certificate Renewal*: The issuance of a new certificate to the subject without changing the subject public key or other information (except the serial number and certificate validity dates) in the certificate.
- *Certificate Update*: The issuance of a new certificate, due to changes in the information in the certificate other than the subject public key.
- *Certificate Rekey*: The generation of a new key pair for the subject and the issuance of a new certificate that certifies the new public key.

The KMIP Specification supports certificate renewals using the Re-Certify operation and certificate updates using the Certify operation. Certificate rekey is supported through the submission of a Re-key Key Pair operation, which generates a replacement (new) key pair, followed by a Certify operation, which issues a new certificate containing the replacement (new) public key.

3.37 Certificate Revocation Lists

Any Certificate Revocation List (CRL) checking which may be required for certificate-related operations such as Register and Re-key should be performed by the client prior to requesting the operation from a server. The server may also choose to perform CRL checks before acting on the certificate-related operation.

3.38 Delegated Login

The Delegated Login operation supports a client delegating a subset of client rights. The operation allows specification of a time limit and/or a request usage count, along with a list of operations and a list of objects that the returned login ticket is permitted to use.

3.39 Discover Versions

The Discover Versions operation allows clients and servers to identify a KMIP protocol version that both client and server understand. The operation addresses both the “dumb” and “smart” client scenarios. Dumb clients may simply pick the first protocol version that is returned by the server, assuming that the client provides the server with a list of supported protocol version. Smart clients may request the server to return a complete list of supported protocol versions by sending an empty request payload and picking a protocol version that is supported by both client and server.

Clients specify the protocol version in the request header and optionally provide a list of protocol versions in the request payload. If the protocol version in the request header is not specified in the request payload and the server does not support any protocol version specified in the request payload, the server returns an empty list in the response payload. In this scenario, clients are aware that the request did not result in an error and could communicate with the server using the protocol version specified in the request header.

3.40 Interop

During formal KMIP interoperability testing it is helpful for a client to be able to indicate which test case is being executed to enable a server to denote this within its logs and use the information for automated comparison against expected results.

3.41 Login and Logout

The Login and Logout operations support caching of authentication checks to enable more efficient interaction with authentication services. The Login operation allows specification of a time limit or request usage count for the returned login ticket.

3.42 Query

Query is defined as both a Client-to-Server and Server to Client operation. Conceptually, the Client-to-Server Query and Server to Client Query operations are equivalent, but the parameters used in each of these queries may differ since some concepts are only applicable to a server or to a client.

3.42.1 Query Function

Function values allow a KMIP client to obtain a wide range of information about a KMIP server and allow a KMIP client to determine which extensions a KMIP server supports (Query Extension List and Query Extension Map); which random number generators a KMIP server supports (Query RNGs); which external validations (e.g., FIPS140, Common Criteria) apply to the KMIP server (Query Validations); which KMIP Profiles from each version of the KMIP specification a KMIP server supports (Query Profiles); which optional areas of the KMIP specification and alternate behaviors a KMIP server supports (Query Capabilities); and which methods of automatic client credential registration (Query Client Registration Methods) are supported by the KMIP server.

Test cases for using these various Query functions may be found in the KMIP Test Case document [\[KMIP-TC\]](#).

3.43 Registering a Key Pair

During a Create Key Pair or Re-key Key Pair operation, a Private Key Link and Public Key Link attribute is automatically created by the server (i.e., a link is created from the private key to the public key and vice versa). Certain attributes are the same for both objects and are set by the server while creating the key pair. The KMIP protocol does not support an equivalent capability for registering a key pair. Clients can register the objects independently and manually set the Public Key Link or Private Key Length attributes to make the server aware that these keys are associated with each other. When the Public Key Link or Private Key Length attribute is set for either object, the server should verify that the registered objects indeed correspond to each other.

Clients should perform the following steps when registering a key pair:

1. Register the public key and set all associated attributes:
 - a. Cryptographic Algorithm
 - b. Cryptographic Length
 - c. Cryptographic Usage Mask
 - d. *Private Key Link* referencing the corresponding Private Key by Unique Identifier or by Name
2. Register the private key and set all associated attributes:
 - a. Cryptographic Algorithm is the same for both public and private key
 - b. Cryptographic Length is the same for both public and private key
 - c. Cryptographic Parameters may be set; if set and no Cryptographic Parameters attribute is present on the public key, the value is the same for both the public and private key
 - d. Cryptographic Usage Mask is set, but does not contain the same value for both the public and private key
 - e. *Public Key Link* referencing the corresponding Public Key by Unique Identifier or by Name

3.44 Using Notify and Put Operations

The Notify and Put operations are operations in the KMIP protocol that are initiated by the server, rather than the client. As client-initiated requests are able to perform these functions (e.g., by polling to request notification), these operations are optional for conforming KMIP implementations. However, they provide a mechanism for optimized communication between KMIP servers and clients.

In using Notify and Put, the following constraints and guidelines should be observed:

- The client enrolls with the server, so that the server knows how to locate the client to which a Notify or Put is being sent and which events for the Notify are supported. However, such registration is outside the scope of the KMIP protocol. Registration also includes a specification of whether a given client supports Put and Notify, and what attributes may be included in a Put for a particular client.
- Communication between the client and the server is authenticated. Authentication for a particular client/server implementation is at a minimum accomplished using one of the mandatory authentication mechanisms (see [\[KMIP-PROF\]](#)). Further strengthening of the client/server communications integrity by means of signed message content and/or wrapped keys is recommended.
- In order to minimize possible divergence of key or state information between client and server as a result of server-initiated communication, any client receiving Notify or Put messages returns acknowledgements of these messages to the server. This acknowledgement may be at communication layers below the KMIP layer, such as by using transport-level acknowledgement provided in TCP/IP.

For client devices that are incapable of responding to messages from the server, communication with the server happens via a proxy entity that communicates with the server, using KMIP, on behalf of the client. It is possible to secure communication between a proxy entity and the client using other, potentially proprietary mechanisms.

3.45 Flow Control

Flow Control or "Endpoint Role" is useful for "Put" and "Notify" operations for clients that are available intermittently or are not directly accessible to the server or where the client must form the network-level and TLS-level connection but the KMIP message flow needs to be inverted. It provides a uniform means of delivering data from the server to the client that would be available by requests from a polling client.

3.46 Client and Server Correlation Values

The Client Correlation Value can be used by a client to provide additional, non-critical information to a server. The value need not be unique. The server should log the value and be able to identify log records that contain a given Client Correlation Value. An example use of the Client Correlation value is to track the KMIP test case number used when performing interoperability testing.

The Server Correlation Value is generated by a server and used to uniquely identify each request. The value should be globally unique and should be logged by the server for each request. The server provides the value to the client within the response to each KMIP operation and the client should store the value in its log. Ideally both the client and server should locate log records that contain a given Server Correlation Value so that the workflow associated with each request may be traced across client and server.

3.47 Reducing Multiple Requests Through the Use of Batch

KMIP supports batch operations in order to reduce the number of calls between the client and server. For example, Locate and Get are likely to be commonly accomplished within a single batch request.

KMIP does not require that batch operations are atomic on the server side but it does allow for such a capability to be provided by a server. If servers implement such atomicity, the client is able to use the "undo" mode to request roll-back for batch operations implemented as atomic transactions.

A client can determine if a server supports the optional "undo" or the optional "continue" mode using a Query operation with the Query Function value of Query Capability Information and by checking the value of the Batch Undo Capability or Batch Continue Capability fields in the response.

3.48 Maximum Message Size

When a server is processing requests in a batch, it should compare the cumulative response size of the message to be returned after each request with the specified Maximum Response Size. If the message is too large, it should prepare a maximum message size error response message at that point, rather than

continuing with operations in the batch. This increases the client's ability to understand what operations have and have not been completed.

When processing individual requests within the batch, the server that has encountered a Maximum Response Size error should not return attribute values or other information as part of the error response.

The Locate operation also supports the concept of a maximum item count to include in the returned list of unique identifiers.

3.49 Message Extensions

Any number of vendor-specific extensions may be included in Message Extension optional structures. This allows KMIP implementations to create multiple extensions to the protocol.

3.50 Result Message Text

KMIP specifies the Result Status, the Result Reason and the Result Message as normative message contents. For the Result Status and Result Reason, the enumerations provided in [\[KMIP-SPEC\]](#) are the normative values. The values for the Result Message text are implementation-specific. In consideration of internationalization, it is recommended that any vendor implementation of KMIP provide appropriate language support for the Return Message. How a client specifies the language for Result Messages is outside the scope of the KMIP.

3.51 Result Reasons

Additional Result Reasons have been added to enable a client to self-diagnose error situations with additional information about the nature of the error. Previously many common error conditions resulted in a single Result Reason. More specificity improves the adoption of additional capabilities within the specification.

3.52 Passing Attestation Data

In some scenarios the server may want assurance of the integrity of the client's system before honoring a client's request. Additionally, the server may want a guarantee of the freshness of the attestation computation in the integrity measurement.

Generally, the process takes four passes:

1. The client sends a request to the server which requires attestation.
2. The server returns a random nonce to the client that will be used in the attestation computation to guarantee the freshness of the measurement.
3. The client sends a request to the server which includes the measurement of the client's system, and the measurement contains the nonce from the server.
4. The server verifies the measurement and sends the appropriate response to the client.

Passing attestation data with a client request can be achieved in KMIP as follows:

1. The client sends a request to the server with the Attestation Capable Indicator set to True in the request header.
2. If the request requires attestation, the server will return an "Attestation Required" error with a Nonce object in the response header. If the client request fails for any reason other than "Attestation Required", the server will not include a nonce in the error message.
3. The client uses the nonce received from the server in the attestation computation that will be used in the measurement.
 - a. The client forms an Attestation Credential Object which contains either the measurement from the client or an assertion from a third party if the server is not capable or willing to verify the attestation data from the client.

- b. The client then issues a request which contains the Attestation Credential Object in the request header.
4. The server validates the measurement or assertion data in the Credential Object, checks that the nonce in the Credential Object matches one sent recently by the server, then sends the appropriate response to complete the request issued by the client. If the measurement or assertion data in the Credential Object does not validate or if the nonce does not match one sent recently by the server, the server will return an "Attestation Failed" error instead of completing the request issued by the client.

The server needs to be capable of processing and verifying multiple Credential Objects in the same request header since Attestation Credentials do not provide the same type of authentication as the Username and Password or Device Credential.

How frequently (e.g. every request, every 100 requests, etc.) the server generates a new random nonce depends on server policy. The lifetime of the nonce once the server has sent it to the client (i.e., the timeframe in which the client must return the nonce before needs to request a fresh nonce from the server) also depends on server policy.

If the client sends a request that requires attestation, but the client has not set the Attestation Capable Indicator to True, then the server will send a "Permission Denied" error and will not include a Nonce object in the response header.

3.53 Vendor Extensions

KMIP allows for vendor extensions in a number of areas:

1. Enumerations have specific ranges which are noted as extensions
2. Item Tag values of the form 0x54xxxx are reserved for vendor extensions
3. Attributes may be defined by the client with a "x-" prefix or by the server with a "y-" prefix

Extensions may be used by vendors to communicate information between a KMIP client and a KMIP server that is not currently defined within the KMIP specification.

A common use of extensions is to allow for the structured definition of attributes using KMIP TTLV encoding rather than encoding vendor specific information in opaque byte strings.

3.54 Counters

A server-applied integer applied to cryptographic objects (a counter type per operation type).

A Counter is incremented by the server upon successful completion of the listed operation performed using the applicable object. The simplest example is where a symmetric key is used to perform an Encrypt operation. In this case the Encrypt Counter for that Symmetric Key object would be incremented once the operation is successfully completed.

One of the use cases envisaged for Counters is to aid in security policy automation. They allow an implementer to (for example) apply specific behaviors when a given counter value is reached (i.e., deactivation of a key once its usage count is exceeded).

3.55 PKCS#11

KMIP v2.0 added support for serialization of PKCS#11 API calls over a KMIP network transport. This enables cross-vendor interoperable PKCS#11 consumer implementations and PKCS#11 aware security gateways to operate without requiring knowledge of vendor-proprietary protocols.

KMIP v3.0 adds support for expressing the serialization format in XML rather than binary for the purpose of test case development. The underlying protocol remains the defined binary encoding in the "PKCS#11 Profiles" but for ease of test case development, the XML representation defined by the OASIS PKCS#11 Technical Committee can be used for test cases.

The PKCS#11 Profiles [[KMIP-PROF](#)] have been updated to show both the binary serialization and the XML representation.

3.56 Post Quantum Cryptography

Quantum computers pose a threat to modern (aka classic) cryptography which is foundational to many cybersecurity capabilities used today. Cryptography is based on complex mathematical problems which are difficult to solve even with the most sophisticated of computers. But some of these problems will be trivial to solve with a sufficiently capable quantum computer (aka Cryptographically Relevant Quantum Computer (CRQC)). Public key (asymmetric) cryptographic algorithms (e.g., RSA, ECDSA, ECDH) used for digital signatures and key agreement are especially vulnerable and need to be replaced with new quantum-resistant or Post Quantum Cryptography (PQC) algorithms.

While it is not known when a Cryptographically Relevant Quantum Computer (CRQC) will exist, research and advancements in quantum computing technology continues to proceed. Threat actors are aware of these advancements and the vulnerability of modern cryptography and are actively capturing and recording encrypted communications and data in the hopes that they can use a quantum computer to break the keys and decrypt the data in the future (i.e., *Harvest Now, Decrypt Later* Attack).

In response to this emerging threat, the U.S. National Institute of Standards and Technology (NIST) via the [Post Quantum Cryptography Standardization Project](#) has been leading efforts since 2016 to standardize quantum resistant algorithms that are secure against both quantum and classic attacks.

In August 2024, NIST publish standards for three additional quantum resistant algorithms namely:

- [FIPS 203: Module Lattice-Based Key Encapsulation Mechanism \(ML-KEM\) Standard](#)
- [FIPS 204: Module Lattice-Based Digital Signature \(ML-DSA\) Standard](#)
- [FIPS 205: Stateless Hash-Based Digital Signature \(SLH-DSA\) Standard](#)

ML-KEM, ML-DSA and SLH-DSA are just the first set of quantum-resistant algorithms that will be standardized and implemented. While standardization efforts have selected algorithms that address a broad number of cryptographic use cases, the current set of algorithms do not address all potential use cases. Great care has been taken to analyze the security of these algorithms, but there is always the possibility that they could be broken in the future and will need to be replaced. There are also concerns about the mathematical diversity of these algorithms which are either hash-based or lattice-based. In light of these considerations, NIST and other countries around the world are continuing their programs to identify, assess and standardize additional algorithms. Expect the introduction of additional quantum-resistant algorithms over the next decade which address additional cryptographic use cases, provide enhanced security and performance and are more mathematically diverse.

Standardization of the quantum-resistant cryptographic algorithms themselves is only the beginning. Considerable standardization efforts are required to adapt existing cryptographic capabilities and protocols to use the quantum-resistant cryptographic algorithms and facilitate implementation at scale.

Within the TLS protocol, adoption of a hybrid key exchange approach (using both classical and post-quantum algorithms) has rapidly occurred. The Quantum Safe Algorithm Suite within KMIP 3.0 mandates the use of these hybrid KEMs to defend against the *Harvest Now, Decrypt Later* attack.

Migration of cryptographic based capabilities and protocols to quantum-resistant algorithms will have a pervasive impact and will take time. Historically, single algorithms replacements (e.g., SHA-1 to SHA-2, DES/3DES to AES) have taken 10+ years to complete. Migration to quantum-resistant algorithms is significantly more complex. It involves replacement of multiple (asymmetric) algorithms. The quantum-resistant algorithms are very different (e.g., longer key size, longer signature size, different performance) from the classic algorithms they are replacing. Hence they are not drop-in replacements but will require effort to properly integrate them into cryptographic based capabilities and protocols.

4 Applying KMIP Functionality

This section describes how to apply the functionality described in the Key Management Interoperability Protocol Specification to address specific key management usage scenarios or to solve key management related issues.

4.1 Locating Keys in Specific States

It is possible to formulate Locate queries to address any of the following conditions:

- Exact match of a transition to a given state. Locate the key(s) with a transition to a certain state at a specified time (t).
- Range match of a transition to a given state. Locate the key(s) with a transition to a certain state at any time at or between two specified times (t and t').
- Exact match of a state at a specified time. Locate the key(s) that are in a certain state at a specified time (t).
- Match of a state during an entire time range. Locate the key(s) that are in a certain state during an entire time specified with times (t and t'). Note that the Activation Date could occur at or before t and that the Deactivation Date could occur at or after t'+1.
- Match of a state at some point during a time range. Locate the key(s) that are in a certain state at some time at or between two specified times (t and t'). In this case, the transition to that state could be before the start of the specified time range.

This is accomplished by allowing any date/time attribute to be present either once (for an exact match) or at most twice (for a range match).

For instance, if the state we are interested in is Active, the Locate queries would be the following (corresponding to the list above):

- Exact match of a transition to a given state: Locate (ActivationDate(t)). Locate keys with an Activation Date of t.
- Range match of a transition to a given state: Locate (ActivationDate(t), ActivationDate(t')). Locate keys with an Activation Date at or between t and t'.
- Exact match of a state at a specified time: Locate (ActivationDate (0), ActivationDate (t), DeactivationDate (t+1), DeactivationDate (MAX_INT), CompromiseDate (t+1), CompromiseDate (MAX_INT)). Locate keys in the Active state at time t, by looking for keys with a transition to Active before or until t, and a transition to Deactivated or Compromised after t (because we don't want the keys that have a transition to Deactivated or Compromised before t). The server assumes that keys without a DeactivationDate or CompromiseDate is equivalent to MAX_INT (i.e., infinite).
- Match of a state during an entire time range: Locate (ActivationDate (0), ActivationDate (t), DeactivationDate (t'+1), DeactivationDate (MAX_INT), CompromiseDate (t'+1), CompromiseDate (MAX_INT)). Locate keys in the Active state during the entire time from t to t'.
- Match of a state at some point during a time range: Locate (ActivationDate (0), ActivationDate (t'-1), DeactivationDate (t+1), DeactivationDate (MAX_INT), CompromiseDate (t+1), CompromiseDate (MAX_INT)). Locate keys in the Active state at some time from t to t', by looking for keys with a transition to Active between 0 and t'-1 and exit out of Active on or after t+1.

The queries would be similar for Initial Date, Deactivation Date, Compromise Date and Destroy Date.

In the case of the Destroyed-Compromise state, there are two dates recorded: the Destroy Date and the Compromise Date. For this state, the Locate operation would be expressed as follows:

- Exact match of a transition to a given state: Locate (CompromiseDate (t), State (Destroyed-Compromised)) and Locate (DestroyDate (t), State (Destroyed-Compromised)). KMIP does not support the OR in the Locate request, so two requests should be issued. Locate keys that were Destroyed and transitioned to the Destroyed-Compromised state at time t, and locate keys that were Compromised and transitioned to the Destroyed-Compromised state at time t.

- Range match of a transition to a given state: Locate (CompromiseDate (t), CompromiseDate (t'), State (Destroyed-Compromised)) and Locate (DestroyDate (t), DestroyDate (t'), State (Destroyed-Compromised)). Locate keys that are Destroyed-Compromised and were Compromised or Destroyed at or between t and t'.
- Exact match of a state at a specified time: Locate (CompromiseDate (0), CompromiseDate (t), DestroyDate (0), DestroyDate (t)); nothing else is needed, since there is no exit transition. Locate keys with a Compromise Date at or before t, and with a Destroy Date at or before t. These keys are, therefore, in the Destroyed-Compromised state at time t.
- Match of a state during an entire time range: Locate (CompromiseDate (0), CompromiseDate (t), DestroyDate (0), DestroyDate (t)). Same as above. As there is no exit transition from the Destroyed-Compromised state, the end of the range (t') is irrelevant.
- Match of a state at some point during a time range: Locate (CompromiseDate (0), CompromiseDate (t'-1), DestroyDate (0), DestroyDate (t'-1)). Locate keys with a Compromise Date at or before t'-1, and with a Destroy Date at or before t'-1. As there is no exit transition from the Destroyed-Compromised state, the start of the range (t) is irrelevant.

4.2 Using Wrapped Keys with KMIP

KMIP provides the option to register and get keys in wrapped format. Clients request the server to return a wrapped key by including the Key Wrapping Specification in the Get Request Payload. Similarly, clients register a wrapped key by including the Key Wrapping Data in the Register Request Payload.

The Wrapping Method within the Key Wrapping Data identifies the type of mechanism used to wrap the key but does not identify the algorithm or block cipher mode. It is possible to determine these from the attributes set for the specified Encryption Key or MAC/Signing Key. If a key has multiple Cryptographic Parameters set, clients may include the applicable parameters in Key Wrapping Specification. If omitted, the server chooses the Cryptographic Parameter attribute with the lowest index.

The Key Value includes both the Key Material and, optionally, attributes of the key; these may be provided by the client in the Register Request Payload; the server only includes attributes when requested in the Key Wrapping Specification of the Get Request Payload. The Key Value may be encrypted, signed/MACed, or both encrypted and signed/MACed (and vice versa). In addition, clients have the option to request or import a wrapped Key Block according to standards, such as ANSI TR-31, or vendor-specific key wrapping methods.

It is important to note that if the Key Wrapping Specification is included in the Get Request Payload, the Key Value may not necessarily be encrypted. If the Wrapping Method is MAC/sign, the returned Key Value is in plaintext, and the Key Wrapping Data includes the MAC or Signature of the Key Value.

The Key Wrap Type parameter in the Get operation may be used to determine how a key that was registered as a wrapped key is returned to a client. Specifying a value of Not Wrapped ensures that the server returns the unwrapped key value. A value of As Registered can be used to retrieve the key value as it was provided in the Register operation. In the latter case, the wrapping key need not be known to the server. If no Key Wrap Type is provided, then the server may choose to return the key either wrapped or unwrapped. A Get operation may use both a Key Wrap Type and a Wrapping Key Specification, in which case the Key Wrap Type is processed as if there was no Wrapping Key Specification, and the result is then wrapped as specified.

Prior to wrapping or unwrapping a key, the server should verify that the wrapping key is allowed to be used for the specified purpose. For example, if the Unique ID of a symmetric key is specified in the Key Wrapping Specification inside the Get request, the symmetric key should have the "Wrap Key" bit set in its Cryptographic Usage Mask. Similarly, if the client registers a signed key, the server should verify that the Signature Key, as specified by the client inside the Key Wrapping Data, has the "Verify" bit set in the Cryptographic Usage Mask. If the wrapping key is not permitted to be used for the requested purpose (e.g., when the Cryptographic Usage Mask is not set), the server should return the Operation Failed result status.

4.2.1 Encrypt-only Example with a Symmetric Key as an Encryption Key for a Get Request and Response

The client sends a Get request to obtain a key that is stored on the server. When the client sends a Get request to the server, a Key Wrapping Specification may be included. If a Key Wrapping Specification is included in the Get request, and a client wants the requested key and its Cryptographic Usage Mask attribute to be wrapped with AES key wrap, the client includes the following information in the Key Wrapping Specification:

- Wrapping Method: Encrypt
- Encryption Key Information
 - Unique Key ID: Key ID of the AES wrapping key
 - Cryptographic Parameters: The Block Cipher Mode is NISTKeyWrap (not necessary if default block cipher mode for wrapping key is NISTKeyWrap)
- Attribute Name: Cryptographic Usage Mask

The server uses the Unique Key ID specified by the client to determine the attributes set for the proposed wrapping key. For example, the algorithm of the wrapping key is not explicitly specified inside the Key Wrapping Specification. The server determines the algorithm to be used for wrapping the key by identifying the Algorithm attribute set for the specified Encryption Key.

The Cryptographic Parameters attribute should be specified by the client if multiple instances of the Cryptographic Parameters exist, and the lowest index does not correspond to the NIST key wrap mode of operation. The server should verify that the AES wrapping key has NISTKeyWrap set as an allowable Block Cipher Mode, and that the “Wrap Key” bit is set in the Cryptographic Usage Mask.

If the correct data was provided to the server, and no conflicts exist, the server AES key wraps the Key Value (both the Key Material and the Cryptographic Usage Mask attribute) for the requested key with the wrapping key specified in the Encryption Key Information. The wrapped key (byte string) is returned in the server's response inside the Key Value of the Key Block.

The Key Wrapping Data of the Key Block in the Get Response Payload includes the same data as specified in the Key Wrapping Specification of the Get Request Payload except for the Attribute Name.

4.2.2 Encrypt-only Example with a Symmetric Key as an Encryption Key for a Register Request and Response

The client sends a Register request to the server and includes the wrapped key and the Unique ID of the wrapping key inside the Request Payload. The wrapped key is provided to the server inside the Key Block. The Key Block includes the Key Value Type, the Key Value, and the Key Wrapping Data. The Key Value Type identifies the format of the Key Material, the Key Value consists of the Key Material and optional attributes that may be included to cryptographically bind the attributes to the Key Material, and the Key Wrapping Data identifies the wrapping mechanism and the encryption key used to wrap the object and the wrapping mechanism.

Similar to the example in 4.2.1 the key is wrapped using the AES key wrap. The Key Value includes four attributes: Cryptographic Algorithm, Cryptographic Length, Cryptographic Parameters, and Cryptographic Usage Mask.

The Key Wrapping Data includes the following information:

- Wrapping Method: Encrypt
- Encryption Key Information
 - Unique Key ID: Key ID of the AES wrapping key
 - Cryptographic Parameters: The Block Cipher Mode is NISTKeyWrap (not necessary if default block cipher mode for wrapping key is NISTKeyWrap)

Attributes do not need to be specified in the Key Wrapping Data. When registering a wrapped Key Value with attributes, clients may include these attributes inside the Key Value without specifying them inside the Template-Attribute.

Prior to unwrapping the key, the server determines the wrapping algorithm from the Algorithm attribute set for the specified Unique ID in the Encryption Key Information. The server verifies that the wrapping key may be used for the specified purpose. In particular, if the client includes the Cryptographic Parameters in the Encryption Key Information, the server verifies that the specified Block Cipher Mode is set for the wrapping key. The server also verifies that the wrapping key has the “Unwrap Key” bit set in the Cryptographic Usage Mask.

The Register Response Payload includes the Unique ID of the newly registered key and an optional list of attributes that were implicitly set by the server.

4.2.3 Encrypt-only Example with an Asymmetric Key as an Encryption Key for a Get Request and Response

The client sends a Get request to obtain a key (either symmetric or asymmetric) that is stored on the server. When the client sends a Get request to the server, a Key Wrapping Specification may be included. If a Key Wrapping Specification is included, and the key is to be wrapped with an RSA public key using the OAEP encryption scheme, the client includes the following information in the Key Wrapping Specification. Note that for this example, attributes for the requested key are not requested.

- Wrapping Method: Encrypt
- Encryption Key Information
 - Unique Key ID: Key ID of the RSA public key
 - Cryptographic Parameters:
 - Padding Method: OAEP
 - Hashing Algorithm: SHA-256

The Cryptographic Parameters attribute is specified by the client if multiple instances of Cryptographic Parameters exist for the wrapping key, and the lowest index does not correspond to the associated padding method. The server should verify that the specified Cryptographic Parameters in the Key Wrapping Specification and the “Wrap Key” bit in the Cryptographic Usage Mask are set for the corresponding wrapping key.

The Key Wrapping Data returned by the server in the Key Block of the Get Response Payload includes the same data as specified in the Key Wrapping Specification of the Get Request Payload.

The Mask Generator and Mask Generator Hashing Algorithm enumerations allow the Mask Generation Function (MGF) and associated algorithm to be explicitly specified. If Mask Generator Hashing Algorithm is omitted, then the value specified for Hashing Algorithm is assumed.

The example above assumes the same algorithm (SHA-256) is used for both the Mask Generation Function (MGF) and hashing data.

4.2.4 MAC-only Example with an HMAC Key as an Authentication Key for a Get Request and Response

The client sends a Get request to obtain a key that is stored on the server. When the client sends a Get request to the server, a Key Wrapping Specification may be included. If a key and Custom Attribute (i.e., x-Nonce) is to be MACed with HMAC SHA-256, the following Key Wrapping Specification is specified:

- Wrapping Method: MAC/sign
- MAC/Signature Key Information
 - Unique Key ID: Key ID of the MACing key (note that the algorithm associated with this key would be HMAC-SHA256)
- Attribute Name: x-Nonce

For HMAC, no Cryptographic Parameters need to be specified, since the algorithm, including the hash function, may be determined from the Algorithm attribute set for the specified MAC Key. The server should verify that the HMAC key has the “MAC Generate” bit set in the Cryptographic Usage Mask. Note that an HMAC key does not require the “Wrap Key” bit to be set in the Cryptographic Usage Mask.

The server creates an HMAC value over the Key Value if the specified MACing key may be used for the specified purpose and no conflicts exist. The Key Value is returned in plaintext, and the Key Block includes the following Key Wrapping Data:

- Wrapping Method: MAC/sign
- MAC/Signature Key Information
- Unique Key ID: Key ID of the MACing key
- MAC/Signature: HMAC result of the Key Value

In the example, the custom attribute x-Nonce was included to help clients, who are relying on the proxy model, to detect replay attacks. End-clients, who communicate with the key management server, may not support TLS and may not be able to rely on the message protection mechanisms provided by a security protocol. An alternative approach for these clients would be to use the custom attribute to hold a random number, counter, nonce, date, or time. The custom attribute needs to be created before requesting the server to return a wrapped key and is recommended to be set if clients frequently wrap/sign the same key with the same wrapping/signing key.

4.2.5 Registering a Wrapped Key as an Opaque Cryptographic Object

Clients may want to register and store a wrapped key on the server without the server being able to unwrap the key (i.e., the wrapping key is not known to the server). Instead of storing the wrapped key as an opaque object, clients have the option to store the wrapped key inside the Key Block as an opaque cryptographic object, i.e., the wrapped key is registered as a managed cryptographic object, but the encoding of the key is unknown to the server. Registering an opaque cryptographic object allows clients to set all the applicable attributes that apply to cryptographic objects (e.g., Cryptographic Algorithm and Cryptographic Length),

Opaque cryptographic objects are set by specifying the following inside the Key Block structure:

- Key Format Type: Opaque
- Key Material: Wrapped key as a Byte String

The Key Wrapping Data does not need to be specified.

4.2.6 Encoding Option for Wrapped Keys

KMIP provides the option to specify the Encoding Option inside the Key Wrapping Specification and Key Wrapping Data. This option allows users to Get or Register the Key Value in a non-TTLV encoded format. This may be desirable in a proxy environment, where the end-client is not KMIP-aware.

The Encoding Option is only available if no attributes are specified inside the Key Value. The server returns the Encoding Option Error if both the Encoding Option and Attribute Names are specified inside the Key Wrapping Specification. Similarly, the server is expected to return the Encoding Option Error when registering a wrapped object with attributes inside the Key Value and the Encoding Option is set in the Key Wrapping Data. If no Encoding Option is specified, KMIP assumes that the Key Value is TTLV-encoded. Thus, by default, the complete TTLV-encoded Key Value content, as shown in the example below, is wrapped:

```
Key Material || Byte String || Length || Key Material Value
420043      || 08          || 00000010 || 0123456789ABCDEF0123456789ABCDEF
```

Some end-clients may not understand or have the space for anything more than the actual key material (i.e., 0123456789ABCDEF0123456789ABCDEF in the above example). To wrap only the Key Material value during a Get operation, the Encoding Option (00001 for no encoding) should be specified inside the Key Wrapping Specification. The same Encoding Option should be specified in the Key Wrapping Data when returning the non-TTLV encoded wrapped object inside the Get Response Payload or when registering a wrapped object in non-TTLV encoded format.

It is important to be aware of the risks involved when excluding the attributes from the Key Value. Binding the attributes to the key material in certain environments is essential to the security of the end-client. An untrusted proxy could change the attributes (provided separately via the Get Attributes operation) that determine how the key is being used (e.g., Cryptographic Usage). Including the attributes inside the Key

Value and cryptographically binding it to the Key Material could prevent potential misuse of the cryptographic object and may prevent a replay attack if, for example, a nonce is included as a custom attribute. The exclusion of attributes and therefore the usage of the Encoding Option are only recommended in at least one of the following scenarios:

1. End-clients are registered with the KMIP server and are communicating with the server directly (i.e., the TLS connection is between the server and client).
2. The environment is controlled and non-KMIP-aware end-clients are aware how wrapped cryptographic objects (possibly Raw keys) from the KMIP server should be used without having to rely on the attributes provided by the Get Attributes operation.
3. The wrapped cryptographic object consists of attributes inside the Key Material value. These attributes are not interpreted by the KMIP server but are understood by the end-client. This may be the case if the Key Format Type is opaque or vendor-specific.
4. The proxy communicating with the KMIP server on behalf of the end-client is considered to be trusted and is operating in a secure environment.

Registering a wrapped object without attributes is not recommended in a proxy environment unless scenario 4 is met.

4.3 Interoperable Key Naming for Tape

This section describes methods and provides examples for creating and storing key identifiers that are interoperable across multi-vendor KMIP clients, using the KMIP Tape Library Profiles.

4.3.1 Native Tape Encryption by a KMIP Client

A common method for naming and retrieving keys is needed to support moving tape cartridges between 2 or more KMIP-compliant tape libraries that are all registered with the same KMIP key manager.

4.3.1.1 Method Overview

The method uses the KMIP Tape Library Profile. This profile specifies use of the KMIP Application Specific Information (ASI) attribute. The method supports both client-generated and server-generated key identifiers.

The key identifier is a KMIP string, composed of hexadecimal numeric characters. This string of characters is unique within a chosen namespace. Methods of generating the string are determined by policy. The LIBRARY-LTO namespace is preferred for maximum interoperability.

A compressed (numeric) transformation of the identifier string is stored in the tape format's Key Associated Data. This allows for future retrieval of the key for decryption.

Interoperability is achieved by a) standardized algorithms to map byte values between the numeric (KAD) and text (ASI) representations of the identifier; and b) standardized ordering of bytes within the KAD so the identifier can be re-assembled in the correct sequence by other compliant implementations.

Examples of the algorithms are provided below.

4.3.1.2 Definitions

Key Associated Data (KAD): Part of the tape format. May be segmented into authenticated and unauthenticated fields. KAD usage is detailed in the SCSI SSC-3 standard from the T10 organization.

Application Specific Information (ASI): A KMIP attribute.

Hexadecimal numeric characters: Case-sensitive, printable, single byte ASCII characters representing the numbers 0 through 9 and uppercase alpha A through F. (US-ASCII characters 30h-39h and 41h-46h).

Hexadecimal numeric characters are always paired, each pair representing a single 8-bit numeric value. A leading zero character is provided, if necessary, so that every byte in the tape's KAD is represented by exactly two hexadecimal numeric characters.

$N(k)$: The number of bytes in the tape format combined KAD fields (both authenticated and unauthenticated).

$N(a)$, $N(u)$: The number of bytes in the tape formats authenticated, and unauthenticated KAD fields, respectively.

4.3.1.3 Implementation Example of Algorithm 1. Key identifier string to numeric direction (Converting the ASI string to tape format's KAD)

Refer to the KMIP Tape profile for algorithm 1.

This algorithm is associated with writing the KAD, typically to allow future retrieval of a key. An example implementation is as follows.

1. The client creates a key identifier or obtains one from the server. The identifier is a KMIP string of hexadecimal numeric characters. Copy the string to an input buffer of size $2*N(k)$ bytes. For LTO4, an 88-character string is sufficient to represent any key name stored directly in the KAD fields. For LTO5, a 184-character string is sufficient to represent any key name stored directly in the KAD fields.
2. Define output buffers for unauthenticated KAD, and authenticated KAD, of size $N(u)$ and $N(a)$ respectively. For LTO4, this would be 32 bytes of unauthenticated data, and 12 bytes of authenticated data. For LTO5, this would be 32 bytes of unauthenticated data and 60 bytes of authenticated data.
3. Define the standard POSIX (also known as C) locale. Each character in the string is a single-byte US-ASCII character.
4. First, populate the authenticated KAD buffer, converting a sub-string consisting of the last (rightmost) $2*N(a)$ characters of the key identifier string.
5. When the authenticated KAD is filled, next populate the unauthenticated KAD buffer, by converting the remaining hexadecimal character pairs (if any) of the identifier string.

4.3.1.4 Implementation Example of Algorithm 2. Numeric to key identifier string direction (Converting tape format's KAD to ASI string)

This algorithm is associated with reading the KAD, typically in preparation for retrieving a key. An example implementation is as follows

1. Define an input buffer sized for $N(k)$. For LTO4, $N(k)$ is 44 bytes (12 bytes authenticated, 32 unauthenticated). For LTO5, $N(k)$ is 92 bytes (60 bytes authenticated, 32 bytes unauthenticated).
2. Define an output buffer sufficient to contain a string with a maximum length of $2*N(k)$ bytes.
3. Define the standard POSIX (also known as C) locale. Each character in the string is a single-byte US-ASCII character.
4. First, copy the tape format's unauthenticated KAD data (if any) to the input buffer. Next, bytes from the authenticated KAD are concatenated, after the unauthenticated bytes. In many implementations the unauthenticated KAD is empty, and in those cases the entire input buffer will be populated with bytes from authenticated KAD.
5. For each byte in the input buffer, convert to US-ASCII as follows:
6. Convert the byte's value to exactly 2 hexadecimal numeric characters, including a leading 0 where necessary. Append these 2 numeric characters to the output buffer, with the high-nibble represented by the left-most hexadecimal numeric character.

4.3.1.5 Usage Example

The following usage example will create a key identifier which can be stored in ASI. The identifier will then be translated for storage into a tape format's KAD, using algorithm 1. Both LTO4 and LTO5 examples of KAD contents are provided.

The reverse translation from KAD bytes to the KMIP key identifier is not shown but would be accomplished via algorithm 2. This re-constructed key identifier string would be used to Locate the key via ASI.

Example of creating a key identifier. Implementation-specific material is used to generate a key identifier. The content of this material is based on server or client policy. An example of a text string which could be used to generate a KMIP key identifier for tape is as follows.

SN12345**6**_MFR:XYZ INC_BAR12345_TM2013123**4**

This example is a set of 40 characters which will be used to create a KMIP key identifier for use as specified in the KMIP Tape Profile. Every 8th character is bold.

This set of characters is suitable as a key identifier for either LTO4 or LTO5, since it will fit within the smaller 44-character KAD space of LTO4.

The corresponding KMIP key identifier, which is a string of hexadecimal numeric character pairs, is shown below. This string will be stored in ASI Application Data.

53 4E 31 32 33 34 35 **36** 5F 4D 46 52 3A 58 59 **5A** 20 49 4E 43 5F 42 41 **52**

31 32 33 34 35 5F 54 **4D** 32 30 31 33 31 32 33 **34**

Spaces are shown for to improve readability but are NOT part of the ASI string. Every 8th hexadecimal numeric pair is bold.

Note the identifier has exactly 2x more characters than the material used to generate the KMIP key identifier.

Translating the key identifier to KAD bytes (LTO4). The corresponding KAD content, for use with an LTO4 tape cartridge is shown in the following figure.

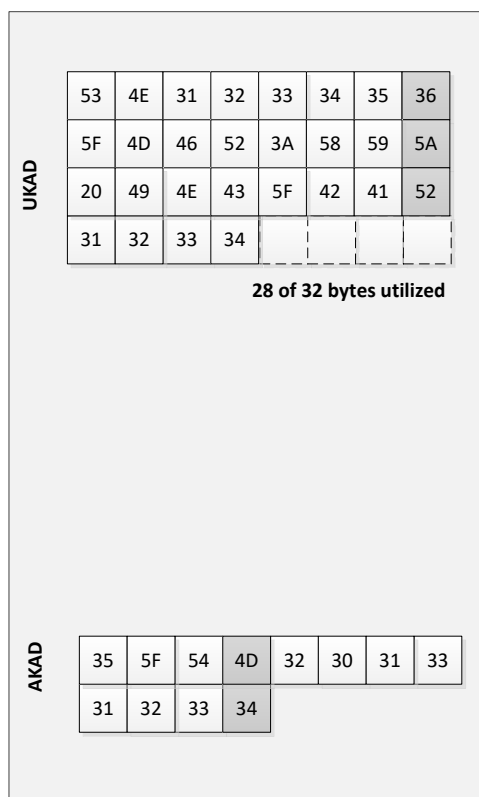


FIGURE 2: KAD CONTENT FOR LTO4

Each square is 1 byte (8 bits). Each square contains the 8-bit value which represents a pair of hexadecimal numeric characters in the KMIP key identifier string.

Every 8th byte of KAD is shaded.

The KAD was populated by converting the rightmost 24 characters (12-character pairs) of the identifier string into bytes of authenticated KAD. The remaining characters of the identifier were written to unauthenticated KAD.

Translating the key identifier to KAD bytes (LTO5). The corresponding KAD for use with an LTO5 and later tape cartridge is shown in the following figure.

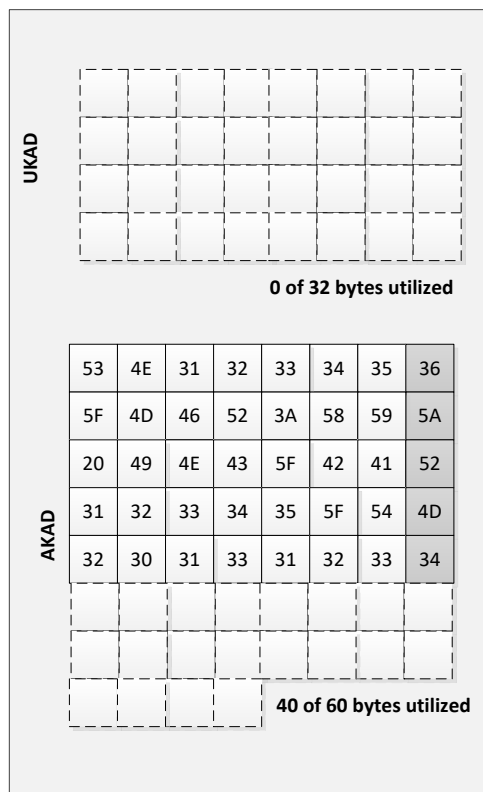


FIGURE 3: KAD CONTENT FOR LTO5

Each square is 1 byte (8 bits). Each square contains the 8-bit value which represents a pair of hexadecimal numeric characters in the key identifier string.

Every 8th byte of KAD is shaded.

The KAD was populated by converting the rightmost 80 characters (40-character pairs) of the identifier string into bytes of authenticated KAD. The unauthenticated KAD is not used because all of the data fits within authenticated KAD.

4.4 Registering Extension Information

As tag values and their interpretation for the most part should be known for a client and server to meaningfully use an extension, the following registration procedure should be used.

1. Document the Extensions including:
 - a. Extension Tag, Extension Name, Extension Type values to be reserved
 - b. A brief description of the purpose of the Extension
 - c. Example use case messages (requests and responses)
 - d. Example Guidance
2. Send the Document to the KMIP TC requesting review
3. Request a KMIP TC ballot on accepting the reservation of the Extension

It is anticipated that a template document may be produced for this registration process.

4.5 Using KMIP for PGP Keys

PGP, both as vendor product and as standard, provides a rich environment for key management that addresses significant use cases related to such areas as secure exchange of email, documents and other resources. Although KMIP is by no means required for support of PGP environments, it can provide a valuable mechanism for movement of PGP keys between a particular PGP environment, such as Symantec Encryption Management Server (SEMS, née PGP Universal), and another key management environment.

KMIP does not attempt to represent the full range of functionality in PGP environments. However, the use cases related to movement of PGP keys across environments, described in the KMIP Use Cases document, can be supported by taking advantage both of the PGP-specific capabilities in KMIP, such as the PGP Key object and of KMIP messages, objects, operations and attributes in general.

The PGP Key managed object contains a PGP key (specified in [\[RFC4880\]](#)) as an opaque blob. KMIP compliant servers do not need to understand the fine structure of PGP keys. The intention here is that PGP-enabled clients be able to discover the PGP Key managed cryptographic objects by searching for one of the various names contained within the block. The Alternative Name attribute can be used to specify one or more names (e.g. User IDs) that are attached to the PGP Key object. The PGP-enabled clients are expected to digest the PGP Key object and properly assign these Alternative Name attributes on to the cryptographic managed object. The KMIP server does not have to do this work.

Internally, PGP keys may contain many public-private key pairs, each tied to a specific type of encryption operations (one key for signing, one for encryption, and one to tie the other two together in a trust relationship is one typical arrangement.) The Link attribute supports new values that enable the description of this set of PGP Key relationships. The new values are parent, child, previous and next. For example, the private and public keys associated with a PGP Key can be pointed to from the PGP Key with the “child” link attribute. Additional Decryption Keys (ADK) can be pointed to from the PGP Key with the “child” link attribute and can be point to each other with the “previous” and “next” link attributes. In this way, the link attributes can be used to define the structural relationships required to establish the web of trust for a PGP Key.

As mentioned above, KMIP does not attempt to represent all the information about PGP keys that would be managed within a PGP implementation. For example, policies such as algorithms supported, by a PGP key are not expressed within KMIP. Instead, KMIP enables the specification of these attributes, if necessary, as information enclosed within the opaque value defined for a given PGP key. This information would be handled by security administration and out-of-band coordination between the PGP environments that participate in the KMIP exchanges related to PGP keys.

KMIP does not support an operation to create PGP Key objects and KMIP compliant servers are not required to support PGP Key object creation. PGP-enabled clients need to do the key creation and pass the resulting information up to KMIP.

4.6 KMIP Client Registration Models

There are several common approaches to registering KMIP clients with KMIP servers:

- Manual client registration within a single trust boundary
- Automatic client registration across multiple trust boundaries
- Configuring a KMIP Server for use with Automatic Client Registration

The goal of these approaches is to establish the KMIP-interoperable secure channel or channels between KMIP servers and clients, such as a mutually-authenticated TLS channel.

In order to support the goal of establishing an interoperable approach to establishing this channel, this section provides more detailed information about these approaches to client registration.

The Query Function, *Client Registration Methods*, provides a client a means to determine which automated client registrations method(s) a KMIP server supports. Although a KMIP server is not required to support any automated registration method. Reflecting common usage for KMIP, all three of the scenarios described below discuss the use of X.509 certificates for trust establishment; other

mechanisms may be used instead but are not described here. Similarly, all three scenarios describe the establishment of a mutually-authenticated TLS connection as the basis trusted exchange of KMIP messages, corresponding to the published KMIP authentication suite profiles; other authentication mechanisms can be used with KMIP, but are not described here.

4.6.1 Manual Client Registration

In this approach, there is no assumption of pre-population of authentication credentials in the client, such as by installing an X.509 certificate into a tape library or drive during the manufacturing process. Rather, a credential is propagated out-of-band to the client administrator, who installs it into the client environment. The credential is then used on initial and subsequent contact between the client and server systems.

This registration model often entails the server administrator creating a package that contains 1) X.509 certificate that the client will use to identify itself to the server when creating a TLS mutually-authenticated session; 2) information about the X.509 certificate that will be presented by the server to the client during negotiation of the mutual authentication, enabling the client to verify the server identity; and 3) possibly additional information that can be included in the credential of the KMIP message sent across the established channel, such as to provide finer granularity for particular drives within a tape library. As indicated, the use of this package of materials takes place during two phases: first during the establishment of the TLS secure channel; second during the transmission of KMIP messages. The server administrator must have configured the server to recognize the X.509 certificate presented by the client, to present the correct X.509 certificate of its own to the client in return and to recognize the additional information provided in the credential object in the KMIP message, if any.

In this model, KMIP is not used to transmit the X.509 certificate and server information used in establishing the secure channel. There is nothing to prevent KMIP being used to send this information; but commonly this is done using mechanisms other than KMIP, nor is there any expectation that KMIP is a required or default mechanism for propagating the credential and the information. The distribution mechanism, therefore, may well vary across vendors.

The use of additional information as the credential in the KMIP message is also neither required nor a default. Inclusion of such a credential in the package distributed to the client administrator and in one or more KMIP messages is also, therefore, likely to vary across vendors.

4.6.2 Automated Client Registration

In this approach the credential used to establish a mutually-authenticated TLS connection is not provided in the package provided by the server administrator. Instead, the establishment of trust between the client and server is accomplished by some other mechanism.

A client may use the Query Function, *Client Registration Methods*, to identify which automated client registrations method(s) a KMIP server supports. Methods included in the enumeration are Unspecified, Server Pre-generated, Server On-demand, Client Generated and Client Registered. Test cases which cover how to use the *Client Registration Method* Query Function and implement the referenced automated client registration methods are included in the KMIP Test Cases document [\[KMIP-TC\]](#).

One example of a Server Pre-generated method is having an X.509 certificate installed in a client device during the manufacturing process. This certificate is then used as a bootstrap mechanism for the subsequent exchange of the kind of information exchanged between client administrator and server administrator.

There will often be configuration activity for the client device based on information, such as a Service ID, received from the server administrator. Once the client administrator initiates auto-registration, the client device sends the X.509 certificate to the server, for example in order to use it to establish an initial TLS session. The server then sends the equivalent of the registration packet in section 4.6.1 above to the client and the client returns the certificate to be used for establishing the secure TLS channel with the server.

In this model, administrator intervention may be required to determine whether the initial client certificate should be accepted. The scenario above assumes that the return of the server's packet of registration is immediate and automatic; alternatively, the return of the packet of information may be done manually by

the server administrator, as in section 4.6.1 above; or the return of the packet of server information may be done by the server, but only after that action has been approved by an administrator.

As discussed in section 4.6.1, KMIP can be used by the client in sending the X.509 certificates to the server. However, this is not required. If it is sent to the server using a KMIP register operation, the server must be able to distinguish that this operation is intended not only to register the cryptographic object, but also to initiate the registration of the client as a legitimate participant in KMIP message exchange.

4.6.3 Registering Sub-Clients Based on a Trusted Primary Client

Another model is to register sub-clients of a trusted client. In this model, the establishment of trust between the client and server can be accomplished using either of the approaches in section 4.6.1 or 4.6.2. However, the server may also send additional information to the client, such as a “tenant identifier”, which it will have to provide to sub-clients for them to use they attempt to register individually. The individual sub-clients would follow a registration model such as that described in section 4.6.2, but would also provide the tenant identifier along with the X.509 certificate so that the server can decide whether to accept the client, based on such criteria as (for example) the TCP/IP address of the sub-client relative to that of the primary client.

This approach can be used for tiered clients that need to be grouped based on their association with a larger trusted entity, but that also need individual identities and trust relationships established based on those identities.

KMIP can be used for sending both the client certificate and the tenant identifier to the server.

4.7 Using One Time Pad Algorithms

A One Time Pad within the Cryptographic Algorithm enumeration allows for One Time Pad algorithms to be used within the context of KMIP. Test cases and examples which demonstrate the usage of the One Time Pad algorithm in combination with the cryptographic operations may be found the KMIP Test Cases document [\[KMIP-TC\]](#).

4.8 Cryptographic Erasure and Key Shredding

Cryptographic Erasure, which is also referred to as Cryptographic (Crypto) Shredding, is the deletion of a cryptographic key in order to render data encrypted with that key unrecoverable. Cryptographic Erasure may be supported with KMIP via the Client-to-Server *Destroy* operation and via the Server-to-Client *Notify* operation.

If key material used to encrypt data is only persisted (stored) at the KMIP server and not at the encryption point (KMIP client is co-located with or a proxy to the encryption point) then cryptographic erasure may be achieved by sending a KMIP *Destroy* request to the KMIP server. How the KMIP server destroys the key and whether the server chooses to retain meta-data about the deleted key is a vendor implementation decision and outside of the scope of KMIP.

If the key material is persisted at the encryption point (in addition to the at the KMIP server) then the KMIP server may use the (optional) *Notify* operation with the *Destroy Date* attribute included to inform the KMIP client (co-customer attributed with or a proxy to the encryption point) that the key has been deleted. Upon receipt of this notification the KMIP Client (and encryption point) could be programmed to destroy any copy of the key that it has stored. Note that KMIP does not require that either a KMIP Server or KMIP client to support the *Notify* operation nor does KMIP require that a KMIP client behave in a specific manner based on a notification. This example just demonstrates how *Notify* could be used to facilitate key destruction and cryptographic shredding. The method the KMIP client uses to destroy the key and whether to maintain meta-data for the key is a vendor implementation and is also out of scope of KMIP.

Key destruction eliminates a cryptographic key so that it is no longer accessible for use in a cryptographic function (e.g. NIST [\[SP800-57-1\]](#) Destroy state). Key shredding takes the destruction process a step further so that the cryptographic key is destroyed using a method where the key is not only inaccessible from the cryptographic application in which the key was used, but that the key is not recoverable using forensics methods outside of the cryptographic application. This type of destruction method normally follows a data sanitization method such as overwriting key with random data (e.g. [\[IEEE 2883-2022\]](#), NIST [\[SP800-88r2\]](#)). KMIP does not impose any requirements on the methods a KMIP client or KMIP

server uses to destroy a key. However, it is possible to query the type of destruction methods that a KMIP client or KMIP server support by using the *Query Capabilities* function within the *Query* operation. The *Capability Information* object which is returned in the Query response may contain the *Shredding Algorithm* which specifies the supported destruction method.

4.9 ReEncrypt

There are contexts in which a client may wish to change the key being used to protect information without having to expose the plaintext to the client.

The Ephemeral tag within the Request Batch item allows the client to request that information is not returned to the client.

When combined with the Data Enumeration, to refer to the output from a previous Batch Item, a client can request a Decryption followed by an Encryption without the plaintext output from the Decryption being returned to the client.

The Ephemeral Enumeration has three options which control what information is not returned – from just omitting the Data field, to just returning the Unique Identifier, to omitting the entire response payload.

4.10 Key Replication Between Servers

Key management systems may be designed to consist of multiple servers to address fault tolerance, performance, management, and communication considerations. Each server would contain the same objects and attributes. Operations made to one server would be replicated to all the other servers.

4.10.1 Architecture Considerations

Some architectures permit operations to be performed even though all servers are not continuously connected. This means that an operation performed on one server may not be immediately replicated to the rest of the servers. Or different operations can be performed on the same key on different servers. Any inconsistencies may not be noticed until the servers can again communicate with one another.

One example of an inconsistency is that a given key may have different states on different servers. One server might consider a key with a given unique identifier to be Active, while a different server might consider the same key with the same unique identifier to be Compromised. Another example of an inconsistency is that two different keys in different servers may end up with the same Name. Forward and backward links might also be inconsistent in different servers.

Key management systems consisting of multiple servers need to be carefully designed to ensure that servers can resolve any inconsistencies that arise, and those systems that claim KMIP compliance must ensure that any resolution complies with the KMIP specification [\[KMIP-SPEC\]](#).

5 Deprecated KMIP Functionality

This section describes KMIP functionality that has been deprecated.

Use of deprecated functionality is discouraged since such functionality may be dropped in a future release of the [\[KMIP-SPEC\]](#).

5.1 KMIP Deprecation Rule

Items in the normative [KMIP Specification \[KMIP-SPEC\]](#) document can be marked deprecated in any document version but will be removed only in a major version. Similarly, conformance clauses or other normative information in the [KMIP Profiles \[KMIP-PROF\]](#) document can be deprecated in any document version but removed only in a major version. Information in the non-normative [KMIP Usage Guide \[this document\]](#) and [KMIP Test Cases \[KMIP-TC\]](#) documents may be removed in any document version.

5.2 Deprecated Functions

There are presently no KMIP 3.x deprecated functions.

Appendix A. Acknowledgments

A.1 Participants

The following individuals were members of this Technical Committee during the creation of this document and their contributions are gratefully acknowledged:

Anthony Berglas, Cryptsoft Pty Ltd.
Justin Corlett, Cryptsoft Pty Ltd.
Nadia Arias Guzman, Cryptsoft Pty Ltd.
Tim Hudson, Cryptsoft Pty Ltd.
Scott Marshall, Cryptsoft Pty Ltd.
Bruce Rich, Cryptsoft Pty Ltd. (formerly IBM)
Greg Scott, Cryptsoft Pty Ltd.
Jason Thatcher, Cryptsoft Pty Ltd.
Judy Furlong, Dell Technologies
Deepak Gaikwad, Dell Technologies
Chandra Nelogal, Dell Technologies
Brett Shank, Fornetix
Gerald Stueve, Fornetix
Charles White, Fornetix
Christopher Hillier, Hewlett Packard Enterprise (HPE)
Kyle Wuolle, KeyNexus Inc
Alka Acharya, IBM
Rinkesh Bansal, IBM
Jesse Sedler, IBM
Jeff Bartell, Individual
Oscar So, Individual
Su Yeol Cho, MDS Intelligence Inc
Sun Ho, MDS Intelligence Inc
Asaf Azarsky, NetApp
Tim Chevalier, NetApp
Michael Coletta, OASIS
Kelly Cullinane, OASIS
Chet Ensign, OASIS
Jane Harnad, OASIS
Alexis Abell, Oracle
Susan Gleeson, Oracle
Mark Joseph, P6R, Inc
Jim Sosoy, P6R, Inc
Kenli Chong, QuintessenceLabs Pty Ltd.
John Leiseboer, QuintessenceLabs Pty Ltd.

Stephen Edwards, Semper Fortis Solutions
Tony Cox, TC Logic (formerly Cryptsoft Pty Ltd.)
Ed Chang, Utimaco IS GmbH
Steven Wierenga, Utimaco IS GmbH

Appendix B. Table of Figures and Tables

Table of Figures

Figure 1: Aggregator Client Example.....	12
Figure 2: KAD Content for LTO4	41
Figure 3: KAD Content for LTO5	42

Table of Tables

Table 1: ID Placeholder Prior to and Resulting from a KMIP Operation	15
Table 2: Cryptographic Usage Masks Pairs	18
Table 3: ECC Recommended Curve Mapping	23

Appendix C. Revision History

Revision	Date	Editor	Changes Made
V3.0-wd01 Draft 1	5 January 2022	Judy Furlong	Initial 3.0 Working Draft Moved KMIP 2.1 UG content into KMIP 3.0 starter document
V3.0-wd01 Draft 2	7 January 2022	Judy Furlong	Incorporate new KMIP 3.0 content Deactivate
V3.0-wd01 Draft 3	28 July 2022	Judy Furlong	Incorporate new KMIP 3.0 content - Split Key (OASIS SAM) - PKCS #11 Version posted to OASIS site.
V3.0-wd02	25 February 2026	Judy Furlong	First phase of incorporating feedback and new 3.0 content from T. Hudson Version posted to OASIS site.
V3.0-wd03	20 May 2026	Judy Furlong	Incorporated feedback on WD02 and earlier, added remainder of 3.0 content and made changes to align with the Spec vs Usage Guide Delta discussion Version posted to OASIS site.
V3.0-cnd01	18 June 2026	Judy Furlong	Updated WD03 to CND01 in preparation for publication and initial public review
V3.0-cnd01a	6 July 2026	Judy Furlong	Editorial – Removed the second Appendix A and fixed the number of Appendices